

GLOBAL
EDITION



Objects First with Java™

*A Practical Introduction
Using BlueJ*

SIXTH EDITION

David J. Barnes
Michael Kölling

ALWAYS LEARNING

PEARSON

Tradução PT-BR - página original 1

Edição Global

Objetos Primeiro com Java™
Uma Introdução Prática Usando BlueJ
SEXTA edição
David J. Barnes
Michael Kölling



David J. Barnes and Michael Kölling

University of Kent

Sixth Edition

Global Edition



Boston Columbus Indianapolis New York San Francisco Hoboken
Amsterdam Cape Town Dubai London Madrid Milan Munich Paris Montreal Toronto
Delhi Mexico City Sao Paulo Sydney Hong Kong Seoul Singapore Taipei Tokyo

Tradução PT-BR - página original 2

David J. Barnes e Michael Kölling
Universidade de Kent
Sexta Edição
Edição Global

Objetos Primeiro com Java™
Uma Introdução Prática Usando BlueJ

Boston Columbus Indianapolis Nova York São Francisco Hoboken
Amsterdã Cidade do Cabo Dubai Londres Madri Milão Munique Paris Montreal Toronto
Delhi Cidade do México São Paulo Sydney Hong Kong Seul Singapura Taipei Tóquio

Vice President, Editorial Director, ECS: *Marcia Horton*
Executive Editor: *Tracy Dunkelberger*
Editorial Assistant: *Kristy Alaura*
Vice President of Marketing: *Christy Lesko*
Director of Field Marketing: *Tim Galligan*
Product Marketing Manager: *Bram Van Kempen*
Field Marketing Manager: *Demetrius Hall*
Marketing Assistant: *Jon Bryant*
Director of Product Management: *Erin Gregg*
Team Lead, Program and Project Management:
Scott Disanno
Program Manager: *Carole Snyder*
Project Manager: *Camille Trentacoste*

Senior Specialist, Program Planning and Support:
Maura Zaldivar-Garcia

Acquisitions Editor, Global Edition: *Sourabh Maheshwari*

Project Editor, Global Edition: *Amrita Naskar*

Senior Media Editor, Global Edition: *Gargi Banerjee*

Senior Manufacturing Controller, Production, Global Edition: *Trudy Kimber*

Cover Designer: *Lumina Datamatics Ltd.*

Cover Art: *brm1949/Fotolia*

Inventory Manager: *Ann Lam*

Media Project Manager: *Leslie Sumrall*

Full-Service Project Management, Composition, and Art:
Chandrasekar Subramanian, SPI Global

Pearson Education Limited
Edinburgh Gate
Harlow
Essex CM20 2JE
England

and Associated Companies throughout the world

Visit us on the World Wide Web at:
www.pearsonglobaleditions.com

© Pearson Education Limited 2017

The rights of David J. Barnes and Michael Kölling to be identified as the authors of this work have been asserted by them in accordance with the Copyright, Designs and Patents Act 1988.

Authorized adaptation from the United States edition, entitled Objects First with Java™: A Practical Introduction Using BlueJ, 6th edition, ISBN 978-0-13-447736-7, by David J. Barnes and Michael Kölling, published by Pearson Education © 2017.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without either the prior written permission of the publisher or a license permitting restricted copying in the United Kingdom issued by the Copyright Licensing Agency Ltd, Saffron House, 6-10 Kirby Street, London EC 1N 8TS.

All trademarks used herein are the property of their respective owners. The use of any trademark in this text does not vest in the author or publisher any trademark ownership rights in such trademarks, nor does the use of such trademarks imply any affiliation with or endorsement of this book by such owners.

Unless otherwise indicated herein, any third-party trademarks that may appear in this work are the property of their respective owners and any references to third-party trademarks, logos or other trade dress are for demonstrative or descriptive purposes only. Such references are not intended to imply any sponsorship, endorsement, authorization, or promotion of Pearson's products by the owners of such marks, or any relationship between the owner and Pearson Education, Inc. or its affiliates, authors, licensees or distributors.

The programs and applications presented in this book have been included for their instructional value. They have been tested with care, but are not guaranteed for any particular purpose. The publisher does not offer any warranties or representations, nor does it accept any liabilities with respect to the programs or applications.

ISBN 10: 1-292-15904-9

ISBN 13: 978-1-292-15904-1

British Library Cataloguing-in-Publication Data

A catalogue record for this book is available from the British Library

10 9 8 7 6 5 4 3 2 1

Printed and bound by Vivar in Malaysia

Tradução PT-BR - página original 3

Pearson Education Limited
Edinburgh Gate
Harlow
Essex CM20 2JE
Inglaterra
e empresas associadas em todo o mundo
Visite-nos na World Wide Web em:
www.pearsonglobal editions.com

© Pearson Education Limited 2017

Os direitos de David J. Barnes e Michael Kölling de serem identificados como autores desta obra foram por eles reivindicados de acordo com o Copyright, Designs and Patents Act de 1988.

Adaptação autorizada da edição dos Estados Unidos, intitulada *Objects First with Java™: A Practical Introduction Using BlueJ*, 6ª edição, ISBN 978-0-13-447736-7, de David J. Barnes e Michael Kölling, publicada pela Pearson Education © 2017.

Todos os direitos reservados. Nenhuma parte desta publicação pode ser reproduzida, armazenada em um sistema de recuperação ou transmitida, de qualquer forma ou por qualquer meio, eletrônico, mecânico, por fotocópia, gravação ou de outra maneira, sem a permissão prévia por escrito da editora ou sem uma licença que permita cópia restrita no Reino Unido, emitida pela Copyright Licensing Agency Ltd, Saffron House, 6-10 Kirby Street, London EC 1N 8TS.

Todas as marcas comerciais utilizadas aqui pertencem aos seus respectivos proprietários. O uso de qualquer marca comercial neste texto não concede ao autor ou à editora direitos de propriedade sobre ela, nem implica qualquer afiliação ou endosso deste livro por seus proprietários.

Salvo indicação em contrário, quaisquer marcas de terceiros que possam aparecer nesta obra pertencem aos seus respectivos proprietários, e referências a marcas, logotipos ou outras identidades visuais de terceiros destinam-se apenas a fins demonstrativos ou descritivos. Tais referências não pretendem sugerir patrocínio, endosso, autorização ou promoção dos produtos da Pearson pelos proprietários dessas marcas, nem qualquer relação entre esses proprietários e a Pearson Education, Inc., suas afiliadas, autores, licenciados ou distribuidores.

Os programas e aplicativos apresentados neste livro foram incluídos por seu valor instrucional. Eles foram testados com cuidado, mas não são garantidos para qualquer finalidade específica. A editora não oferece garantias ou declarações, nem assume responsabilidades relacionadas a esses programas ou aplicativos.

ISBN 10: 1-292-15904-9
ISBN 13: 978-1-292-15904-1
Dados de Catalogação na Publicação da British Library
Um registro de catálogo desta obra está disponível na British Library
10 9 8 7 6 5 4 3 2 1
Impresso e encadernado pela Vivar, na Malásia

Vice-presidente, Diretora Editorial, ECS: Marcia Horton
 Editora Executiva: Tracy Dunkelberger
 Assistente Editorial: Kristy Alaura
 Vice-presidente de Marketing: Christy Lesko
 Diretor de Marketing de Campo: Tim Galligan
 Gerente de Marketing de Produto: Bram Van Kempen
 Gerente de Marketing de Campo: Demetrius Hall
 Assistente de Marketing: Jon Bryant
 Diretora de Gestão de Produto: Erin Gregg
 Líder da Equipe, Gestão de Programas e Projetos: Scott Disanno
 Gerente de Programa: Carole Snyder
 Gerente de Projeto: Camille Trentacoste
 Especialista Sênior, Planejamento e Suporte de Programas: Maura Zaldivar-Garcia
 Editora de Aquisições, Edição Global: Sourabh Maheshwari
 Editora de Projeto, Edição Global: Amrita Naskar
 Editora Sênior de Mídia, Edição Global: Gargi Banerjee
 Controladora Sênior de Produção, Edição Global: Trudy Kimber
 Designer da Capa: Lumina Datamatics Ltd.
 Arte da Capa: brm1949/Fotolia
 Gerente de Estoque: Ann Lam
 Gerente de Projeto de Mídia: Leslie Sumrall
 Gestão Completa do Projeto, Composição e Arte: Chandrasekar Subramanian, SPi Global

À minha esposa Helen,
djb

A K.W.
mk

To my wife Helen,
djb

To K.W.
mk

Esta página foi deixada intencionalmente em branco.

This page intentionally left blank



Foreword	xiv
Preface	xv
List of Projects Discussed in Detail in This Book	xxv
Acknowledgments	xxviii
Part 1 Foundations of Object Orientation	1
Chapter 1 Objects and Classes	3
1.1 Objects and classes	3
1.2 Creating objects	4
1.3 Calling methods	5
1.4 Parameters	6
1.5 Data types	7
1.6 Multiple instances	8
1.7 State	9
1.8 What is in an object?	10
1.9 Java code	11
1.10 Object interaction	12
1.11 Source code	13
1.12 Another example	15
1.13 Return values	15
1.14 Objects as parameters	16
1.15 Summary	17
Chapter 2 Understanding Class Definitions	21
2.1 Ticket machines	21
2.2 Examining a class definition	23
2.3 The class header	25
2.4 Fields, constructors, and methods	26
2.5 Parameters: receiving data	32
2.6 Assignment	34

SUMÁRIO

Prefácio de abertura	xiv
Prefácio	xv
Lista de projetos discutidos em detalhes neste livro	xxv
Agradecimentos	xxviii

Parte 1 - Fundamentos da Orientação a Objetos 1

Capítulo 1 - Objetos e Classes	3
1.1 Objetos e classes	3
1.2 Criando objetos	4
1.3 Chamando métodos	5
1.4 Parâmetros	6
1.5 Tipos de dados	7
1.6 Múltiplas instâncias	8
1.7 Estado	9
1.8 O que há em um objeto?	10
1.9 Código Java	11
1.10 Interação entre objetos	12
1.11 Código-fonte	13
1.12 Outro exemplo	15
1.13 Valores de retorno	15
1.14 Objetos como parâmetros	16
1.15 Resumo	17

Capítulo 2 - Compreendendo Definições de Classes	21
2.1 Máquinas de bilhetes	21
2.2 Examinando uma definição de classe	23
2.3 O cabeçalho da classe	25
2.4 Campos, construtores e métodos	26
2.5 Parâmetros: recebendo dados	32
2.6 Atribuição	34

2.7	Methods	35
2.8	Accessor and mutator methods	36
2.9	Printing from methods	39
2.10	Method summary	42
2.11	Summary of the naïve ticket machine	42
2.12	Reflecting on the design of the ticket machine	43
2.13	Making choices: the conditional statement	45
2.14	A further conditional-statement example	47
2.15	Scope highlighting	48
2.16	Local variables	50
2.17	Fields, parameters, and local variables	51
2.18	Summary of the better ticket machine	53
2.19	Self-review exercises	53
2.20	Reviewing a familiar example	55
2.21	Calling methods	57
2.22	Experimenting with expressions: the Code Pad	59
2.23	Summary	61
Chapter 3	Object Interaction	67
3.1	The clock example	67
3.2	Abstraction and modularization	68
3.3	Abstraction in software	69
3.4	Modularization in the clock example	69
3.5	Implementing the clock display	70
3.6	Class diagrams versus object diagrams	71
3.7	Primitive types and object types	72
3.8	The <code>NumberDisplay</code> class	72
3.9	The <code>ClockDisplay</code> class	80
3.10	Objects creating objects	83
3.11	Multiple constructors	84
3.12	Method calls	84
3.13	Another example of object interaction	88
3.14	Using a debugger	92
3.15	Method calling revisited	96
3.16	Summary	97
Chapter 4	Grouping Objects	101
4.1	Building on themes from Chapter 3	101
4.2	The collection abstraction	102

SUMÁRIO

2.7	Métodos	35
2.8	Métodos acessores e modificadores	36
2.9	Imprimindo a partir de métodos	39
2.10	Resumo sobre métodos	42
2.11	Resumo da máquina de bilhetes ingênua	42
2.12	Refletindo sobre o projeto da máquina de bilhetes	43
2.13	Fazendo escolhas: a instrução condicional	45
2.14	Outro exemplo de instrução condicional	47
2.15	Destaque de escopo	48
2.16	Variáveis locais	50
2.17	Campos, parâmetros e variáveis locais	51
2.18	Resumo da máquina de bilhetes melhorada	53
2.19	Exercícios de autoavaliação	53
2.20	Revedo um exemplo conhecido	55
2.21	Chamando métodos	57
2.22	Experimentando expressões: o Code Pad	59
2.23	Resumo	61
Capítulo 3	Interação entre Objetos	67
3.1	O exemplo do relógio	67
3.2	Abstração e modularização	68
3.3	Abstração em software	69
3.4	Modularização no exemplo do relógio	69
3.5	Implementando o mostrador do relógio	70
3.6	Diagramas de classes versus diagramas de objetos	71
3.7	Tipos primitivos e tipos de objetos	72
3.8	A classe <code>NumberDisplay</code>	72
3.9	A classe <code>ClockDisplay</code>	80
3.10	Objetos criando objetos	83
3.11	Múltiplos construtores	84
3.12	Chamadas de métodos	84
3.13	Outro exemplo de interação entre objetos	88
3.14	Usando um depurador	92
3.15	Revisão das chamadas de métodos	96
3.16	Resumo	97
Capítulo 4	Agrupando Objetos	101
4.1	Desenvolvendo os temas do Capítulo 3	101
4.2	A abstração de coleção	102

SUMÁRIO

4.3 Um organizador de arquivos de música	103
4.4 Usando uma classe de biblioteca	104
4.5 Estruturas de objetos com coleções	107
4.6 Classes genéricas	109
4.7 Numeração dentro de coleções	110
4.8 Reproduzindo os arquivos de música	113
4.9 Processando uma coleção inteira	115
4.10 Iteração indefinida	120
4.11 Melhorando a estrutura - a classe Track	128
4.12 O tipo Iterator	131
4.13 Resumo do projeto music-organizer	135
4.14 Outro exemplo: um sistema de leilões	137
4.15 Resumo	147
Capítulo 5 - Processamento Funcional de Coleções (Avançado)	149
5.1 Uma visão alternativa dos temas do Capítulo 4	149
5.2 Monitorando populações de animais	150
5.3 Um primeiro olhar sobre lambdas	154
5.4 O método forEach das coleções	156
5.5 Streams	158
5.6 Resumo	168
Capítulo 6 - Comportamento Mais Sofisticado	171
6.1 Documentação de classes de biblioteca	172
6.2 O sistema TechSupport	173
6.3 Lendo documentação de classes	178
6.4 Adicionando comportamento aleatório	183
6.5 Pacotes e import	189
6.6 Usando mapas para associações	190
6.7 Usando conjuntos	195
6.8 Dividindo strings	195
6.9 Finalizando o sistema TechSupport	197
6.10 Autoboxing e classes wrapper	199
6.11 Escrevendo documentação de classes	201
6.12 public versus private	204
6.13 Aprendendo sobre classes por suas interfaces	206
6.14 Variáveis de classe e constantes	211
6.15 Métodos de classe	214
6.16 Executando sem BlueJ	216

	Contents	vii
4.3	An organizer for music files	103
4.4	Using a library class	104
4.5	Object structures with collections	107
4.6	Generic classes	109
4.7	Numbering within collections	110
4.8	Playing the music files	113
4.9	Processing a whole collection	115
4.10	Indefinite iteration	120
4.11	Improving structure—the Track class	128
4.12	The Iterator type	131
4.13	Summary of the music-organizer project	135
4.14	Another example: an auction system	137
4.15	Summary	147
Chapter 5	Functional Processing of Collections (Advanced)	149
5.1	An alternative look at themes from Chapter 4	149
5.2	Monitoring animal populations	150
5.3	A first look at lambdas	154
5.4	The forEach method of collections	156
5.5	Streams	158
5.6	Summary	168
Chapter 6	More-Sophisticated Behavior	171
6.1	Documentation for library classes	172
6.2	The TechSupport system	173
6.3	Reading class documentation	178
6.4	Adding random behavior	183
6.5	Packages and import	189
6.6	Using maps for associations	190
6.7	Using sets	195
6.8	Dividing strings	195
6.9	Finishing the TechSupport system	197
6.10	Autoboxing and wrapper classes	199
6.11	Writing class documentation	201
6.12	Public versus private	204
6.13	Learning about classes from their interfaces	206
6.14	Class variables and constants	211
6.15	Class methods	214
6.16	Executing without BlueJ	216

6.17	Further advanced material	216
6.18	Summary	220
Chapter 7	Fixed-Size Collections—Arrays	223
7.1	Fixed-size collections	223
7.2	Arrays	224
7.3	A log-file analyzer	224
7.4	The for loop	230
7.5	The <i>automaton</i> project	236
7.6	Arrays of more than one dimension (advanced)	244
7.7	Arrays and streams (advanced)	251
7.8	Summary	252
Chapter 8	Designing Classes	255
8.1	Introduction	256
8.2	The <i>world-of-zuul</i> game example	257
8.3	Introduction to coupling and cohesion	259
8.4	Code duplication	260
8.5	Making extensions	263
8.6	Coupling	266
8.7	Responsibility-driven design	270
8.8	Localizing change	273
8.9	Implicit coupling	274
8.10	Thinking ahead	277
8.11	Cohesion	278
8.12	Refactoring	282
8.13	Refactoring for language independence	286
8.14	Design guidelines	291
8.15	Summary	292
Chapter 9	Well-Behaved Objects	295
9.1	Introduction	295
9.2	Testing and debugging	296
9.3	Unit testing within BlueJ	297
9.4	Test automation	304
9.5	Refactoring to use with streams (advanced)	311
9.6	Debugging	312
9.7	Commenting and style	314
9.8	Manual walkthroughs	315

SUMÁRIO	
6.17	Material avançado adicional 216
6.18	Resumo 220
Capítulo 7 - Coleções de Tamanho Fixo - Arrays 223	
7.1	Coleções de tamanho fixo 223
7.2	Arrays 224
7.3	Um analisador de arquivos de log 224
7.4	O laço for 230
7.5	O projeto automaton 236
7.6	Arrays com mais de uma dimensão (avançado) 244
7.7	Arrays e streams (avançado) 251
7.8	Resumo 252
Capítulo 8 - Projetando Classes 255	
8.1	Introdução 256
8.2	O exemplo do jogo world-of-zuul 257
8.3	Introdução a acoplamento e coesão 259
8.4	Duplicação de código 260
8.5	Fazendo extensões 263
8.6	Acoplamento 266
8.7	Projeto orientado a responsabilidades 270
8.8	Localizando mudanças 273
8.9	Acoplamento implícito 274
8.10	Pensando adiante 277
8.11	Coesão 278
8.12	Refatoração 282
8.13	Refatoração para independência de linguagem 286
8.14	Diretrizes de projeto 291
8.15	Resumo 292
Capítulo 9 - Objetos Bem Comportados 295	
9.1	Introdução 295
9.2	Testes e depuração 296
9.3	Testes unitários no BlueJ 297
9.4	Automação de testes 304
9.5	Refatoração para uso com streams (avançado) 311
9.6	Depuração 312
9.7	Comentários e estilo 314
9.8	Inspeções manuais passo a passo 315

	Contents	ix
9.9	Print statements	320
9.10	Debuggers	324
9.11	Debugging streams (advanced)	325
9.12	Choosing a debugging strategy	326
9.13	Putting the techniques into practice	327
9.14	Summary	327
Part 2	Application Structures	329
Chapter 10	Improving Structure with Inheritance	331
10.1	The <i>network</i> example	331
10.2	Using inheritance	343
10.3	Inheritance hierarchies	345
10.4	Inheritance in Java	346
10.5	<i>Network</i> : adding other post types	349
10.6	Advantages of inheritance (so far)	351
10.7	Subtyping	352
10.8	The <code>Object</code> class	358
10.9	The collection hierarchy	359
10.10	Summary	360
Chapter 11	More about Inheritance	363
11.1	The problem: <i>network</i> 's display method	363
11.2	Static type and dynamic type	365
11.3	Overriding	368
11.4	Dynamic method lookup	370
11.5	<code>super</code> call in methods	373
11.6	Method polymorphism	374
11.7	Object methods: <code>toString</code>	374
11.8	Object equality: <code>equals</code> and <code>hashCode</code>	377
11.9	Protected access	379
11.10	The <code>instanceof</code> operator	381
11.11	Another example of inheritance with overriding	382
11.12	Summary	385
Chapter 12	Further Abstraction Techniques	389
12.1	Simulations	389
12.2	The foxes-and-rabbits simulation	390
12.3	Abstract classes	405

SUMÁRIO

9.9	Instruções de impressão	320
9.10	Depuradores	324
9.11	Depurando streams (avançado)	325
9.12	Escolhendo uma estratégia de depuração	326
9.13	Colocando as técnicas em prática	327
9.14	Resumo	327
Parte 2 - Estruturas de Aplicações 329		
Capítulo 10 - Melhorando a Estrutura com Herança 331		
10.1	O exemplo network	331
10.2	Usando herança	343
10.3	Hierarquias de herança	345
10.4	Herança em Java	346
10.5	Network: adicionando outros tipos de postagem	349
10.6	Vantagens da herança (até aqui)	351
10.7	Subtipagem	352
10.8	A classe Object	358
10.9	A hierarquia de coleções	359
10.10	Resumo	360
Capítulo 11 - Mais sobre Herança 363		
11.1	O problema: o método display de network	363
11.2	Tipo estático e tipo dinâmico	365
11.3	Sobrescrita	368
11.4	Busca dinâmica de métodos	370
11.5	Chamada super em métodos	373
11.6	Polimorfismo de métodos	374
11.7	Métodos de Object: toString	374
11.8	Igualdade de objetos: equals e hashCode	377
11.9	Acesso protected	379
11.10	O operador instanceof	381
11.11	Outro exemplo de herança com sobrescrita	382
11.12	Resumo	385
Capítulo 12 - Técnicas Adicionais de Abstração 389		
12.1	Simulações	389
12.2	A simulação foxes-and-rabbits	390
12.3	Classes abstratas	405

12.4	More abstract methods	412
12.5	Multiple inheritance	414
12.6	Interfaces	417
12.7	A further example of interfaces	425
12.8	The <code>Class</code> class	427
12.9	Abstract class or interface?	427
12.10	Event-driven simulations	428
12.11	Summary of inheritance	429
12.12	Summary	430

Chapter 13 Building Graphical User Interfaces 433

13.1	Introduction	433
13.2	Components, layout, and event handling	434
13.3	AWT and Swing	435
13.4	The <code>ImageViewer</code> example	435
13.5	<code>ImageViewer 1.0</code> : the first complete version	447
13.6	<code>ImageViewer 2.0</code> : improving program structure	461
13.7	<code>ImageViewer 3.0</code> : more interface components	467
13.8	Inner classes	471
13.9	Further extensions	476
13.10	Another example: <code>MusicPlayer</code>	478
13.11	Summary	481

Chapter 14 Handling Errors 483

14.1	The <i>address-book</i> project	484
14.2	Defensive programming	488
14.3	Server error reporting	491
14.4	Exception-throwing principles	495
14.5	Exception handling	501
14.6	Defining new exception classes	508
14.7	Using assertions	510
14.8	Error recovery and avoidance	513
14.9	File-based input/output	516
14.10	Summary	527

Chapter 15 Designing Applications 529

15.1	Analysis and design	529
15.2	Class design	536
15.3	Documentation	538

SUMÁRIO

12.4	Mais métodos abstratos	412
12.5	Herança múltipla	414
12.6	Interfaces	417
12.7	Outro exemplo de interfaces	425
12.8	A classe <code>Class</code>	427
12.9	Classe abstrata ou interface?	427
12.10	Simulações orientadas a eventos	428
12.11	Resumo sobre herança	429
12.12	Resumo	430
Capítulo 13 - Construindo Interfaces Gráficas com o Usuário 433		
13.1	Introdução	433
13.2	Componentes, layout e tratamento de eventos	434
13.3	AWT e Swing	435
13.4	O exemplo <code>ImageViewer</code>	435
13.5	<code>ImageViewer 1.0</code> : a primeira versão completa	447
13.6	<code>ImageViewer 2.0</code> : melhorando a estrutura do programa	461
13.7	<code>ImageViewer 3.0</code> : mais componentes de interface	467
13.8	Classes internas	471
13.9	Extensões adicionais	476
13.10	Outro exemplo: <code>MusicPlayer</code>	478
13.11	Resumo	481
Capítulo 14 - Tratando Erros 483		
14.1	O projeto <i>address-book</i>	484
14.2	Programação defensiva	488
14.3	Relato de erros do servidor	491
14.4	Princípios de lançamento de exceções	495
14.5	Tratamento de exceções	501
14.6	Definindo novas classes de exceção	508
14.7	Usando assertions	510
14.8	Recuperação e prevenção de erros	513
14.9	Entrada/saída baseada em arquivos	516
14.10	Resumo	527
Capítulo 15 - Projetando Aplicações 529		
15.1	Análise e projeto	529
15.2	Projeto de classes	536
15.3	Documentação	538

SUMÁRIO

15.4 Cooperação 539
15.5 Prototipação 539
15.6 Crescimento do software 540
15.7 Usando padrões de projeto 542
15.8 Resumo 548

Capítulo 16 - Um Estudo de Caso 551
16.1 O estudo de caso 551
16.2 Análise e projeto 552
16.3 Projeto de classes 556
16.4 Desenvolvimento iterativo 561
16.5 Outro exemplo 570
16.6 Indo além 570

Apêndice A: Trabalhando com um Projeto BlueJ 571
A.1 Instalando o BlueJ 571
A.2 Abrindo um projeto 571
A.3 O depurador do BlueJ 571
A.4 Configurando o BlueJ 571
A.5 Alterando o idioma da interface 572
A.6 Usando a documentação local da API 572
A.7 Alterando os modelos de novas classes 572

Apêndice B: Tipos de Dados em Java 573
B.1 Tipos primitivos 573
B.2 Conversão de tipos primitivos 574
B.3 Tipos de objetos 574
B.4 Classes wrapper 575
B.5 Conversão de tipos de objetos 575

Apêndice C: Operadores 577
C.1 Expressões aritméticas 577
C.2 Expressões booleanas 578
C.3 Operadores de curto-circuito 579

Apêndice D: Estruturas de Controle em Java 581
D.1 Estruturas de controle 581
D.2 Instruções de seleção 581

	Contents	xi
15.4	Cooperation	539
15.5	Prototyping	539
15.6	Software growth	540
15.7	Using design patterns	542
15.8	Summary	548
Chapter 16	A Case Study	551
16.1	The case study	551
16.2	Analysis and design	552
16.3	Class design	556
16.4	Iterative development	561
16.5	Another example	570
16.6	Taking things further	570
Appendix A:	Working with a BlueJ Project	571
A.1	Installing BlueJ	571
A.2	Opening a project	571
A.3	The BlueJ debugger	571
A.4	Configuring BlueJ	571
A.5	Changing the interface language	572
A.6	Using local API documentation	572
A.7	Changing the new class templates	572
Appendix B:	Java Data Types	573
B.1	Primitive types	573
B.2	Casting of primitive types	574
B.3	Object types	574
B.4	Wrapper classes	575
B.5	Casting of object types	575
Appendix C:	Operators	577
C.1	Arithmetic expressions	577
C.2	Boolean expressions	578
C.3	Short-circuit operators	579
Appendix D:	Java Control Structures	581
D.1	Control structures	581
D.2	Selection statements	581

D.3	Loops	583
D.4	Exceptions	585
D.5	Assertions	587
Appendix E: Running Java without BlueJ		589
E.1	Executing without BlueJ	589
E.2	Creating executable .jar files	591
E.3	Developing without BlueJ	591
Appendix F: Using the Debugger		593
F.1	Breakpoints	594
F.2	The control buttons	594
F.3	The variable displays	595
F.4	The Call Sequence display	596
F.5	The Threads display	596
Appendix G: JUnit Unit-Testing Tools		597
G.1	Enabling unit-testing functionality	597
G.2	Creating a test class	597
G.3	Creating a test method	597
G.4	Test assertions	598
G.5	Running tests	598
G.6	Fixtures	598
Appendix H: Teamwork Tools		599
H.1	Server setup	599
H.2	Enabling teamwork functionality	599
H.3	Sharing a project	599
H.4	Using a shared project	599
H.5	Update and commit	600
H.6	More information	600
Appendix I: Javadoc		601
I.1	Documentation comments	601
I.2	BlueJ support for javadoc	603
Appendix J: Program Style Guide		605
J.1	Naming	605
J.2	Layout	605

SUMÁRIO

D.3	Laços	583
D.4	Exceções	585
D.5	Assertions	587
Apêndice E: Executando Java sem BlueJ 589		
E.1	Executando sem BlueJ	589
E.2	Criando arquivos .jar executáveis	591
E.3	Desenvolvendo sem BlueJ	591
Apêndice F: Usando o Depurador 593		
F.1	Pontos de interrupção	594
F.2	Botões de controle	594
F.3	Exibições de variáveis	595
F.4	Exibição da sequência de chamadas	596
F.5	Exibição das threads	596
Apêndice G: Ferramentas de Teste Unitário JUnit 597		
G.1	Habilitando a funcionalidade de testes unitários	597
G.2	Criando uma classe de teste	597
G.3	Criando um método de teste	597
G.4	Asserções de teste	598
G.5	Executando testes	598
G.6	Fixtures	598
Apêndice H: Ferramentas de Trabalho em Equipe 599		
H.1	Configuração do servidor	599
H.2	Habilitando a funcionalidade de trabalho em equipe	599
H.3	Compartilhando um projeto	599
H.4	Usando um projeto compartilhado	599
H.5	Update e commit	600
H.6	Mais informações	600
Apêndice I: Javadoc 601		
I.1	Comentários de documentação	601
I.2	Suporte do BlueJ ao javadoc	603
Apêndice J: Guia de Estilo de Programas 605		
J.1	Nomenclatura	605
J.2	Layout	605

SUMÁRIO

J.3 Documentação 606
J.4 Restrições de uso da linguagem 607
J.5 Idiomas/padrões de código 608

Apêndice K: Classes Importantes da Biblioteca 609
K.1 O pacote java.lang 609
K.2 O pacote java.util 610
K.3 Os pacotes java.io e java.nio.file 611
K.4 O pacote java.util.function 612
K.5 O pacote java.net 612
K.6 Outros pacotes importantes 613

Índice 615

	Contents	xiii
J.3	Documentation	606
J.4	Language-use restrictions	607
J.5	Code idioms	608
Appendix K:	Important Library Classes	609
K.1	The java.lang package	609
K.2	The java.util package	610
K.3	The java.io and java.nio.file packages	611
K.4	The java.util.function package	612
K.5	The java.net package	612
K.6	Other important packages	613
Index		615



Foreword

by James Gosling, creator of Java

Watching my daughter Kate and her middle-school classmates struggle through a Java course using a commercial IDE was a painful experience. The sophistication of the tool added significant complexity to the task of learning. I wish that I had understood earlier what was happening. As it was, I wasn’t able to talk to the instructor about the problem until it was too late. This is exactly the sort of situation for which BlueJ is a perfect fit.

BlueJ is an interactive development environment with a mission: it is designed to be used by students who are learning how to program. It was designed by instructors who have been in the classroom facing this problem every day. It’s been refreshing to talk to the folks who developed BlueJ: they have a very clear idea of what their target is. Discussions tended to focus more on what to leave out, than what to throw in. BlueJ is very clean and very targeting.

Nonetheless, this book isn’t about BlueJ. It is about programming.

In Java.

Over the past several years Java has become widely used in the teaching of programming. This is for a number of reasons. One is that Java has many characteristics that make it easy to teach: it has a relatively clean definition; extensive static analysis by the compiler informs students of problems early on; and it has a very robust memory model that eliminates most “mysterious” errors that arise when object boundaries or the type system are compromised. Another is that Java has become commercially very important.

This book confronts head-on the hardest concept to teach: objects. It takes students from their very first steps all the way through to some very sophisticated concepts.

It manages to solve one of the stickiest questions in writing a book about programming: how to deal with the mechanics of actually typing in and running a program. Most books silently skip over the issue, or touch it lightly, leaving the instructor with the burden of figuring out how to relate the book’s material to the actual steps that students have to go through to solve the exercises. Instead, this book assumes the use of BlueJ and is able to integrate the tasks of understanding the concepts with the mechanics of how students can explore them.

I wish it had been around for my daughter last year. Maybe next year . . .

PREFÁCIO DE ABERTURA
por James Gosling, criador do Java

Ver minha filha Kate e seus colegas do ensino fundamental lutando para acompanhar um curso de Java usando uma IDE comercial foi uma experiência dolorosa. A sofisticação da ferramenta acrescentava uma complexidade significativa à tarefa de aprender. Eu gostaria de ter percebido mais cedo o que estava acontecendo. Naquela ocasião, só consegui conversar com o professor sobre o problema quando já era tarde demais. Esse é exatamente o tipo de situação para a qual o BlueJ se encaixa perfeitamente.

O BlueJ é um ambiente de desenvolvimento interativo com uma missão: foi projetado para ser usado por estudantes que estão aprendendo a programar. Ele foi desenvolvido por professores que enfrentam esse problema diariamente em sala de aula. Foi muito agradável conversar com as pessoas que desenvolveram o BlueJ: elas têm uma ideia muito clara de seu objetivo. As discussões tendiam a se concentrar mais no que deveria ficar de fora do que no que deveria ser acrescentado. O BlueJ é muito limpo e muito focado em seu propósito.

Apesar disso, este livro não é sobre o BlueJ. É sobre programação.

Em Java.

Ao longo dos últimos anos, Java tornou-se amplamente utilizado no ensino de programação. Há diversas razões para isso. Uma delas é que Java possui muitas características que facilitam seu ensino: apresenta uma definição relativamente limpa; a análise estática abrangente realizada pelo compilador informa os estudantes sobre problemas logo no início; e possui um modelo de memória muito robusto, que elimina a maior parte dos erros "misteriosos" que surgem quando os limites entre objetos ou o sistema de tipos são violados. Outra razão é que Java adquiriu grande importância comercial.

Este livro enfrenta diretamente o conceito mais difícil de ensinar: objetos. Ele conduz os estudantes desde os primeiros passos até conceitos bastante sofisticados.

O livro também consegue resolver uma das questões mais difíceis ao escrever uma obra sobre programação: como lidar com os aspectos mecânicos de realmente digitar e executar um programa. A maioria dos livros simplesmente ignora essa questão ou a aborda apenas superficialmente, deixando para o professor o trabalho de descobrir como relacionar o conteúdo do livro às etapas reais que os estudantes precisam executar para resolver os exercícios. Em vez disso, este livro pressupõe o uso do BlueJ e consegue integrar a tarefa de compreender os conceitos com a mecânica pela qual os estudantes podem explorá-los.

Eu gostaria que ele já existisse no ano passado, quando minha filha fez o curso. Talvez no próximo ano...

PREFÁCIO

Novidades da sexta edição

Esta é a sexta edição deste livro e, como sempre acontece em uma nova edição, o conteúdo foi adaptado aos desenvolvimentos mais recentes da programação orientada a objetos.

Muitas das mudanças desta vez podem, à primeira vista, ser atribuídas a uma nova versão do Java: Java 8. Essa versão foi lançada em 2014 e atualmente é muito utilizada na prática. Na verdade, sua adoção foi a mais rápida entre todas as novas versões do Java já lançadas; portanto, chegou também o momento de mudar a forma como ensinamos estudantes iniciantes.

As mudanças, porém, vão além da simples adição de algumas novas construções da linguagem. Os aspectos novos mais significativos do Java 8 concentram-se em construções que dão suporte a um estilo de programação funcional (parcial). E é a crescente popularidade da programação funcional que impulsiona essa mudança. A diferença é muito mais profunda e fundamental do que apenas acrescentar uma nova sintaxe. É o renascimento das ideias funcionais na programação moderna em geral - e não apenas a existência do Java 8 - que torna oportuno abordar esses aspectos em uma edição moderna de um livro de programação.

As ideias e técnicas da programação funcional, embora sejam relativamente antigas e bem conhecidas em princípio, tiveram um aumento marcante de popularidade nos últimos anos. Novas linguagens foram desenvolvidas e técnicas funcionais selecionadas passaram a ser incorporadas a linguagens existentes, tradicionalmente imperativas. Uma das principais razões para isso é a mudança no hardware computacional disponível e também a mudança na natureza dos problemas que desejamos resolver.

Quase todas as plataformas de programação atualmente são concorrentes. Até mesmo notebooks de desempenho intermediário e telefones celulares possuem processadores com múltiplos núcleos, tornando o processamento paralelo uma possibilidade real em dispositivos cotidianos. Na prática, porém, isso ainda não acontece em grande escala.

Escrever aplicações que façam uso ideal do processamento concorrente e de múltiplos processadores é muito, muito difícil. A maioria das aplicações disponíveis hoje não explora o hardware atual nem de perto na medida do que seria teoricamente possível.

Isso não deve mudar muito: a oportunidade (e o desafio) representada pelo hardware paralelo continuará existindo, e programar esses dispositivos com linguagens imperativas tradicionais não ficará mais fácil.

É aqui que a programação funcional entra em cena.

Preface

New to the sixth edition

This is the sixth edition of this book, and—as always with a new edition—the content has been adapted to the latest developments in object-oriented programs.

Many of the changes this time can, on the surface, be attributed to a new version of Java: Java 8. This version was released in 2014 and is now very widely used in practice. In fact, it is the fastest adoption of any new Java version ever released; so it is time also to change the way we teach novice students.

The changes are, however, more than merely the addition of a few new language constructs. The most significant new aspects in Java 8 center around new constructs to support a (partial) functional programming style. And it is the growing popularity of functional programming that is driving this change. The difference is much deeper, and much more fundamental, than just adding new syntax. And it is the renaissance of the functional ideas in modern programming generally—not only the existence of Java 8—that makes it timely to cover these aspects in a modern edition of a programming textbook.

The ideas and techniques of functional programming, while fairly old and well known in principle, have seen a marked boost of popularity in recent years, with new languages being developed and selected functional techniques being incorporated into existing, traditionally imperative languages. One of the primary reasons for this is the change in computing hardware available, and also the changing nature of problems we wish to tackle.

Almost all programming platforms now are concurrent. Even mid-range laptops and mobile phones now have processors with multiple cores, making parallel processing a real possibility on everyday devices. But, in practice this is not happening on a large scale.

Writing applications that make optimal use of concurrent processing and multiple processors is very, very difficult. Most applications available today do not exploit current hardware to a degree approaching anything that is theoretically possible.

This is not going to change much: the opportunity (and challenge) of parallel hardware will remain, and programming these devices with traditional imperative languages will not get any easier.

This is where functional programming enters the picture.

With functional language constructs, it is possible to automate some concurrency very efficiently. Programs can potentially make use of multiple cores without much effort on the side of the programmer. Functional constructs have other advantages—more elegant expression for certain problems and often clearer readability—but it is the ability to deal with parallelism that will ensure that functional aspects of programming are going to stay with us for a long time to come.

Every teacher who wants to prepare their students for the future should give them some understanding of functional aspects as well. Without it, one will no longer be able to become a master programmer. A novice certainly does not have to master all of functional programming, but a basic understanding of what it is—and what we can achieve with it—is rapidly becoming essential.

Exactly when functional techniques should be introduced is an interesting question. We do not believe that there is a single right answer for this; various sequences are possible. Functional programming could be covered as an advanced topic at the end of the traditional corpus of this book, or it could be addressed when we first encounter the topics where it is applicable, as an alternative to the imperative techniques. It could even be covered first.

An additional question is how to treat the traditional style of programming in those areas where functional constructs are now available: should they be replaced, or do both need to be covered?

For this book, we recognize that different teachers will have different constraints and preferences. Therefore, we have designed a structure that—we hope—allows different approaches, depending on the preference of the learner or teacher.

- We have not replaced the “old-style” techniques. We cover the new, functional approach in addition to the existing material. Functional constructs in Java are most prominent when working with collections of objects, and the mastering traditional approach—using loops and explicit iteration—is still essential for any programmer. Not only are there millions of lines of code out there that are written in this style—and will be continued to be written in this style—but there are also specific cases where it is necessary to use these techniques even if one generally favors the new functional constructs. Mastering both is the goal.
- We present the new functional-construct-oriented material in the book where we discuss the problems that these constructs address. For example, we address functional collection processing as soon as we encounter collections.
- Chapters and sections covering this new material are, however, clearly marked as “advanced,” and are structured in a manner that they can safely be skipped on first reading (or left out altogether).
- The previous two points enable different approaches to studying this book: if time permits, it can be read in the sequence it is presented, covering the full scope of material—including functional approaches as alternatives to imperative ones—as the problems are encountered which they address. If time is short, these advanced sections can be skipped, and emphasis can be placed on a thorough grounding in imperative, object-oriented programming. (We should emphasize that *functional* is not a contradiction to *object-oriented*: whether the functional material is included in the study, or the course emphasis

PREFÁCIO

Com construções de linguagem funcionais, é possível automatizar parte da concorrência com muita eficiência. Os programas podem potencialmente utilizar múltiplos núcleos sem grande esforço por parte do programador. As construções funcionais têm outras vantagens - permitem expressar determinados problemas de forma mais elegante e muitas vezes tornam o código mais legível -, mas é a capacidade de lidar com paralelismo que garantirá que os aspectos funcionais da programação permaneçam conosco por muito tempo.

Todo professor que deseja preparar seus estudantes para o futuro deve proporcionar a eles também alguma compreensão dos aspectos funcionais. Sem isso, já não será possível tornar-se um programador realmente completo. Um iniciante certamente não precisa dominar toda a programação funcional, mas uma compreensão básica do que ela é - e do que podemos alcançar com ela - está rapidamente se tornando essencial.

O momento exato em que as técnicas funcionais devem ser introduzidas é uma questão interessante. Não acreditamos que exista uma única resposta correta; várias sequências são possíveis. A programação funcional pode ser abordada como um tópico avançado ao final do conteúdo tradicional deste livro, pode ser apresentada quando encontramos pela primeira vez os assuntos em que ela é aplicável, como alternativa às técnicas imperativas, ou pode até mesmo ser ensinada primeiro.

Outra questão é como tratar o estilo tradicional de programação nas áreas em que agora existem construções funcionais: ele deve ser substituído, ou ambos os estilos precisam ser estudados?

Neste livro, reconhecemos que professores diferentes terão restrições e preferências diferentes. Por isso, projetamos uma estrutura que - esperamos - permita abordagens distintas, de acordo com a preferência do estudante ou do professor.

- Não substituímos as técnicas de "estilo antigo". Apresentamos a nova abordagem funcional além do material já existente. As construções funcionais em Java aparecem com maior destaque ao trabalhar com coleções de objetos, mas dominar a abordagem tradicional - usando laços e iteração explícita - continua sendo essencial para qualquer programador. Não apenas existem milhões de linhas de código escritas nesse estilo - e que continuarão sendo escritas assim -, como também existem casos específicos em que essas técnicas são necessárias mesmo quando se prefere, de modo geral, as novas construções funcionais. O objetivo é dominar ambas.

- Apresentamos no livro o novo material orientado a construções funcionais justamente nos pontos em que discutimos os problemas que essas construções resolvem. Por exemplo, tratamos do processamento funcional de coleções assim que encontramos coleções.

- Os capítulos e seções que cobrem esse novo material, entretanto, estão claramente marcados como "avançados" e foram estruturados de modo que possam ser ignorados com segurança em uma primeira leitura (ou até mesmo omitidos completamente).

- Os dois pontos anteriores permitem diferentes formas de estudar este livro: se houver tempo, ele pode ser lido na sequência apresentada, cobrindo todo o material - incluindo abordagens funcionais como alternativas às imperativas - à medida que surgem os problemas que elas resolvem. Se o tempo for curto, essas seções avançadas podem ser puladas e a ênfase pode ficar em uma base sólida de programação imperativa orientada a objetos. Devemos enfatizar que programação funcional não é uma contradição à orientação a objetos: independentemente de o material funcional ser incluído no estudo ou de o curso enfatizar principalmente técnicas imperativas, todo leitor deste livro desenvolverá uma boa compreensão de orientação a objetos.

is largely on imperative techniques, every reader of this book will emerge with a good understanding of object orientation!) Yet another way to approach the material is to skip the advanced sections initially, and cover them as a separate unit at a later time. They present alternative approaches to other constructs and can be covered independently.

We hope this makes clear that this book provides flexibility where readers want it, but also guidance where a reader has no clear preference: just read it in the sequence it is written.

Apart from the major changes described so far, this edition also presents numerous minor improvements. The overall structure, tone, and approach of the book is unchanged; it has worked very well in the past, and there is no reason to deviate from it. However, we continuously re-evaluate and seek to improve where we see opportunities. We now have almost 15 years of continuous experience teaching with this book, and this is reflected in the many minor improvements throughout.

This book is an introduction to object-oriented programming for beginners. The main focus of the book is general object-oriented and programming concepts from a software engineering perspective.

While the first chapters are written for students with no programming experience, later chapters are suitable for more advanced or professional programmers as well. In particular, programmers with experience in a non-object-oriented language who wish to migrate their skills into object orientation should also be able to benefit from the book.

We use two tools throughout the book to enable the concepts introduced to be put into practice: the Java programming language and the Java development environment BlueJ.

Java

Java was chosen because of both its language design and its popularity. The Java programming language itself provides a clean implementation of most of the important object-oriented concepts, and serves well as an introductory teaching language. Its popularity ensures an immense pool of support resources.

In any subject area, having a variety of sources of information available is very helpful, for teachers and students alike. For Java in particular, countless books, tutorials, exercises, compilers, environments, and quizzes already exist, in many different kinds and styles. Many of them are online and many are available free of charge. The huge amount of high quality support material makes Java an excellent choice as an introduction to object-oriented programming.

With so much Java material already available, is there still room for more to be said about it? We think there is, and the second tool we use is one of the reasons . . .

BlueJ

BlueJ deserves much comment. This book is unique in its completely integrated use of the BlueJ environment.

BlueJ is a Java development environment that is being developed and maintained by the Computing Education Research Group at the University of Kent in Canterbury, UK,

PREFÁCIO

Outra maneira de abordar o material é inicialmente pular as seções avançadas e estudá-las posteriormente como uma unidade separada. Elas apresentam abordagens alternativas a outras construções e podem ser estudadas de forma independente.

Esperamos que isso deixe claro que o livro oferece flexibilidade quando os leitores desejam escolher uma abordagem, mas também oferece orientação quando não há uma preferência definida: nesse caso, basta lê-lo na sequência em que foi escrito.

Além das mudanças principais descritas até aqui, esta edição também apresenta inúmeras melhorias menores. A estrutura geral, o tom e a abordagem do livro permanecem inalterados; eles funcionaram muito bem no passado e não há motivo para abandoná-los. No entanto, reavaliamos continuamente o material e procuramos melhorá-lo sempre que vemos uma oportunidade. Agora temos quase 15 anos de experiência contínua ensinando com este livro, e isso se reflete nas muitas pequenas melhorias distribuídas por toda a obra.

Este livro é uma introdução à programação orientada a objetos para iniciantes. Seu foco principal está nos conceitos gerais de programação e orientação a objetos sob a perspectiva da engenharia de software.

Embora os primeiros capítulos sejam escritos para estudantes sem experiência em programação, os capítulos posteriores também são adequados a programadores mais avançados ou profissionais. Em especial, programadores que tenham experiência em uma linguagem não orientada a objetos e desejem migrar suas habilidades para a orientação a objetos também poderão se beneficiar do livro.

Usamos duas ferramentas ao longo da obra para permitir que os conceitos apresentados sejam colocados em prática: a linguagem de programação Java e o ambiente de desenvolvimento Java BlueJ.

Java

Java foi escolhida tanto pelo projeto da linguagem quanto por sua popularidade. A própria linguagem Java fornece uma implementação clara da maioria dos conceitos importantes de orientação a objetos e funciona muito bem como linguagem introdutória para ensino. Sua popularidade garante uma enorme quantidade de recursos de apoio.

Em qualquer área de estudo, dispor de várias fontes de informação é muito útil tanto para professores quanto para estudantes. No caso de Java, em particular, já existem incontáveis livros, tutoriais, exercícios, compiladores, ambientes e testes, em muitos estilos e formatos diferentes. Muitos estão disponíveis on-line e muitos podem ser obtidos gratuitamente. A enorme quantidade de material de apoio de alta qualidade torna Java uma excelente escolha para uma introdução à programação orientada a objetos.

Com tanto material sobre Java já disponível, ainda há espaço para dizer algo novo? Acreditamos que sim, e a segunda ferramenta que utilizamos é uma das razões...

BlueJ

O BlueJ merece muitos comentários. Este livro é único por seu uso completamente integrado do ambiente BlueJ.

BlueJ é um ambiente de desenvolvimento Java desenvolvido e mantido pelo Computing Education Research Group da Universidade de Kent, em Canterbury, Reino Unido,

explicitly as an environment for teaching introductory object-oriented programming. It is better suited to introductory teaching than other environments for a variety of reasons:

- The user interface is much simpler. Beginning students can typically use the BlueJ environment in a competent manner after 20 minutes of introduction. From then on, instruction can concentrate on the important concepts at hand—object orientation and Java—and no time needs to be wasted talking about environments, file systems, class paths, or DLL conflicts.
- The environment supports important teaching tools not available in other environments. One of them is visualization of class structure. BlueJ automatically displays a UML-like diagram representing the classes and relationships in a project. Visualizing these important concepts is a great help to both teachers and students. It is hard to grasp the concept of an object when all you ever see on the screen is lines of code! The diagram notation is a simple subset of UML, tailored to the needs of beginning students. This makes it easy to understand, but also allows migration to full UML in later courses.
- One of the most important strengths of the BlueJ environment is the user’s ability to directly create objects of any class, and then to interact with their methods. This creates the opportunity for direct experimentation with objects, with little overhead in the environment. Students can almost “feel” what it means to create an object, call a method, pass a parameter, or receive a return value. They can try out a method immediately after it has been written, without the need to write test drivers. This facility is an invaluable aid in understanding the underlying concepts and language details.
- BlueJ includes numerous other tools and characteristics that are specifically designed for students of software development. Some are aimed at helping with understanding fundamental concepts (such as the scope highlighting in the editor), some are designed to introduce additional tools and techniques, such as integrated testing using JUnit, or teamwork using a version control system, such as Subversion, once the students are ready. Several of these features are unique to the BlueJ environment.

BlueJ is a full Java environment. It is not a cut-down, simplified version of Java for teaching. It runs on top of Oracle’s Java Development Kit, and makes use of the standard compiler and virtual machine. This ensures that it always conforms to the official and most up-to-date Java specification.

The authors of this book have many years of teaching experience with the BlueJ environment (and many more years without it before that). We both have experienced how the use of BlueJ has increased the involvement, understanding, and activity of students in our courses. One of the authors is also the development lead of the BlueJ system.

Real objects first

One of the reasons for choosing BlueJ was that it allows an approach where teachers truly deal with the important concepts first. “Objects first” has been a battle cry for many textbook authors and teachers for some time. Unfortunately, the Java language does not make this noble goal very easy. Numerous hurdles of syntax and detail have to be overcome before

PREFÁCIO

explicitamente como um ambiente para o ensino introdutório de programação orientada a objetos. Ele é mais adequado ao ensino inicial do que outros ambientes por diversas razões:

- A interface do usuário é muito mais simples. Em geral, estudantes iniciantes conseguem usar o ambiente BlueJ de maneira competente depois de cerca de 20 minutos de introdução. A partir daí, o ensino pode se concentrar nos conceitos realmente importantes - orientação a objetos e Java - sem desperdiçar tempo discutindo ambientes, sistemas de arquivos, class paths ou conflitos de DLLs.

- O ambiente oferece ferramentas didáticas importantes que não estão disponíveis em outros ambientes. Uma delas é a visualização da estrutura de classes. O BlueJ exibe automaticamente um diagrama semelhante a UML que representa as classes e os relacionamentos de um projeto. Visualizar esses conceitos importantes é de grande ajuda para professores e estudantes. É difícil compreender o conceito de objeto quando tudo o que se vê na tela são linhas de código. A notação do diagrama é um subconjunto simples da UML, adaptado às necessidades de estudantes iniciantes. Isso facilita sua compreensão e, ao mesmo tempo, permite uma transição posterior para a UML completa em cursos mais avançados.

- Um dos pontos fortes mais importantes do ambiente BlueJ é a possibilidade de o usuário criar diretamente objetos de qualquer classe e, em seguida, interagir com seus métodos. Isso cria uma oportunidade para experimentar diretamente com objetos, com pouca sobrecarga causada pelo ambiente. Os estudantes quase conseguem "sentir" o que significa criar um objeto, chamar um método, passar um parâmetro ou receber um valor de retorno. Eles podem testar um método imediatamente depois de escrevê-lo, sem precisar criar programas auxiliares de teste. Esse recurso é de valor inestimável para compreender os conceitos subjacentes e os detalhes da linguagem.

- O BlueJ inclui diversas outras ferramentas e características projetadas especificamente para estudantes de desenvolvimento de software. Algumas ajudam na compreensão de conceitos fundamentais, como o destaque de escopo no editor; outras apresentam ferramentas e técnicas adicionais, como testes integrados com JUnit ou trabalho em equipe usando um sistema de controle de versão como Subversion, quando os estudantes estiverem preparados. Diversos desses recursos são exclusivos do ambiente BlueJ.

O BlueJ é um ambiente Java completo. Não é uma versão reduzida ou simplificada de Java criada para ensino. Ele é executado sobre o Java Development Kit da Oracle e utiliza o compilador e a máquina virtual padrão. Isso garante que esteja sempre em conformidade com a especificação Java oficial e mais atualizada.

Os autores deste livro têm muitos anos de experiência de ensino utilizando o ambiente BlueJ - e muitos outros anos de experiência sem ele antes disso. Ambos observaram como o uso do BlueJ aumentou o envolvimento, a compreensão e a atividade dos estudantes em seus cursos. Um dos autores também é o principal responsável pelo desenvolvimento do sistema BlueJ.

Objetos reais primeiro

Uma das razões para escolher o BlueJ foi que ele permite uma abordagem na qual os professores realmente lidam primeiro com os conceitos importantes. "Objetos primeiro" há algum tempo é um lema de muitos autores de livros e professores. Infelizmente, a linguagem Java não torna esse objetivo tão simples. Diversas barreiras de sintaxe e detalhes precisam ser vencidas antes da primeira experiência com um objeto em funcionamento.

the first experience with a living object arises. The minimal Java program to create and call an object typically includes

- writing a class;
- writing a main method, including concepts such as static methods, parameters, and arrays in the signature;
- a statement to create the object (“new”);
- an assignment to a variable;
- the variable declaration, including variable type;
- a method call, using dot notation;
- possibly a parameter list.

As a result, most textbooks typically either

- have to work their way through this forbidding list, and only reach objects somewhere around the fourth chapter; or
- use a “Hello, world”-style program with a single static main method as the first example, thus not creating any objects at all.

With BlueJ, this is not a problem. A student can create an object and call its methods as the very first activity! Because users can create and interact with objects directly, concepts such as classes, objects, methods, and parameters can easily be discussed in a concrete manner before looking at the first line of Java syntax. Instead of explaining more about this here, we suggest that the curious reader dip into Chapter 1—things will quickly become clear then.

An iterative approach

Another important aspect of this book is that it follows an iterative style. In the computing education community, a well-known educational design pattern exists that states that important concepts should be taught early and often.¹ It is very tempting for textbook authors to try and say everything about a topic at the point where it is introduced. For example, it is common, when introducing types, to give a full list of built-in data types, or to discuss all available kinds of loop when introducing the concept of a loop.

These two approaches conflict: we cannot concentrate on discussing important concepts first, and at the same time provide complete coverage of all topics encountered. Our experience with textbooks is that much of the detail is initially distracting, and has the effect of drowning the important points, thus making them harder to grasp.

¹ The “Early Bird” pattern, in J. Bergin: “Fourteen Pedagogical Patterns for Teaching Computer Science,” *Proceedings of the Fifth European Conference on Pattern Languages of Programs* (EuroPLop 2000), Irsee, Germany, July 2000.

PREFÁCIO

O programa Java mínimo para criar e chamar um objeto normalmente inclui:

- escrever uma classe;
- escrever um método main, envolvendo conceitos como métodos static, parâmetros e arrays em sua assinatura;
- uma instrução para criar o objeto (new);
- uma atribuição a uma variável;
- a declaração da variável, incluindo seu tipo;
- uma chamada de método usando a notação de ponto;
- possivelmente uma lista de parâmetros.

Como resultado, a maioria dos livros didáticos normalmente precisa seguir uma de duas alternativas:

- avançar por toda essa lista intimidadora e chegar aos objetos somente por volta do quarto capítulo; ou
- usar como primeiro exemplo um programa no estilo "Hello, world", com um único método static main, sem criar objeto algum.

Com o BlueJ, isso não é um problema. Um estudante pode criar um objeto e chamar seus métodos já como primeira atividade. Como os usuários podem criar objetos e interagir diretamente com eles, conceitos como classes, objetos, métodos e parâmetros podem ser discutidos de maneira concreta antes de observar a primeira linha de sintaxe Java. Em vez de explicar mais sobre isso aqui, sugerimos que o leitor curioso dê uma olhada no Capítulo 1 - tudo ficará claro rapidamente.

Uma abordagem iterativa

Outro aspecto importante deste livro é que ele segue um estilo iterativo. Na comunidade de educação em computação, existe um conhecido padrão de projeto educacional que afirma que conceitos importantes devem ser ensinados cedo e com frequência. É muito tentador para autores de livros tentar dizer tudo sobre um assunto no momento em que ele é apresentado. Por exemplo, ao introduzir tipos, é comum apresentar uma lista completa de todos os tipos de dados incorporados à linguagem; da mesma forma, ao introduzir o conceito de laço, é comum discutir todos os tipos disponíveis de laço.

Essas duas abordagens entram em conflito: não podemos nos concentrar primeiro nos conceitos mais importantes e, ao mesmo tempo, fazer uma cobertura completa de todos os assuntos encontrados. Nossa experiência com livros didáticos mostra que muitos detalhes, no início, apenas distraem e acabam encobrendo os pontos realmente importantes, tornando-os mais difíceis de compreender.

1 O padrão "Early Bird", em J. Bergin: "Fourteen Pedagogical Patterns for Teaching Computer Science", *Proceedings of the Fifth European Conference on Pattern Languages of Programs* (EuroPLop 2000), Irsee, Alemanha, julho de 2000.

In this book we touch on all of the important topics several times, both within the same chapter and across different chapters. Concepts are usually introduced at a level of detail necessary for understanding and applying to the task at hand. They are revisited later in a different context, and understanding deepens as the reader continues through the chapters. This approach also helps to deal with the frequent occurrence of mutual dependencies between concepts.

Some teachers may not be familiar with the iterative approach. Looking at the first few chapters, teachers used to a more sequential introduction will be surprised about the number of concepts touched on this early. It may seem like a steep learning curve.

It is important to understand that this is not the end of the story. Students are not expected to understand everything about these concepts immediately. Instead, these fundamental concepts will be revisited again and again throughout the book, allowing students to get a deeper understanding over time. Since their knowledge level changes as they work their way forward, revisiting important topics later allows them to gain a deeper understanding overall.

We have tried this approach with students many times. Sometimes students have fewer problems dealing with it than some long-time teachers. And remember: a steep learning curve is not a problem as long as you ensure that your students can climb it!

No complete language coverage

Related to our iterative approach is the decision not to try to provide complete coverage of the Java language within the book.

The main focus of this book is to convey object-oriented programming principles in general, not Java language details in particular. Students studying with this book may be working as software professionals for the next 30 or 40 years of their life—it is a fairly safe bet that the majority of their work will not be in Java. Every serious textbook must attempt to prepare them for something more fundamental than the language flavor of the day.

On the other hand, many Java details are essential to actually doing practical programming work. In this book we cover Java constructs in as much detail as is necessary to illustrate the concepts at hand and implement the practical work. Some constructs specific to Java have been deliberately left out of the discussion.

We are aware that some instructors will choose to cover some topics that we do not discuss in detail. However, instead of trying to cover every possible topic ourselves (and thus blowing the size of this book out to 1500 pages), we deal with it using *hooks*. Hooks are pointers, often in the form of questions that raise the topic and give references to an appendix or outside material. These hooks ensure that a relevant topic is brought up at an appropriate time, and leave it up to the reader or the teacher to decide to what level of detail that topic should be covered. Thus, hooks serve as a reminder of the existence of the topic, and as a placeholder indicating a point in the sequence where discussion can be inserted.

Individual teachers can decide to use the book as it is, following our suggested sequence, or to branch out into sidetracks suggested by the hooks in the text.

PREFÁCIO

Neste livro, abordamos todos os tópicos importantes várias vezes, tanto dentro de um mesmo capítulo quanto em capítulos diferentes. Os conceitos normalmente são apresentados com o nível de detalhe necessário para compreender e realizar a tarefa em questão. Mais tarde, eles são revisitados em outro contexto, e a compreensão se aprofunda à medida que o leitor avança pelos capítulos. Essa abordagem também ajuda a lidar com a frequente existência de dependências mútuas entre conceitos.

Alguns professores talvez não estejam familiarizados com a abordagem iterativa. Ao observar os primeiros capítulos, professores acostumados a uma introdução mais sequencial podem se surpreender com a quantidade de conceitos abordados tão cedo. A curva de aprendizagem pode parecer acentuada.

É importante compreender que essa não é a etapa final. Não se espera que os estudantes entendam imediatamente todos os detalhes desses conceitos. Em vez disso, os conceitos fundamentais serão revisitados repetidas vezes ao longo do livro, permitindo que os estudantes desenvolvam uma compreensão mais profunda com o passar do tempo. Como o nível de conhecimento deles muda à medida que avançam, retomar assuntos importantes posteriormente permite uma compreensão global mais profunda.

Experimentamos essa abordagem muitas vezes com estudantes. Às vezes, os próprios estudantes têm menos dificuldades para lidar com ela do que alguns professores com muitos anos de experiência. E lembre-se: uma curva de aprendizagem acentuada não é um problema, desde que você garanta que seus estudantes consigam subi-la.

Sem cobertura completa da linguagem

Relacionada à nossa abordagem iterativa está a decisão de não tentar fazer, neste livro, uma cobertura completa da linguagem Java.

O principal objetivo desta obra é transmitir princípios gerais de programação orientada a objetos, e não detalhes específicos da linguagem Java. Os estudantes que utilizam este livro poderão trabalhar como profissionais de software pelos próximos 30 ou 40 anos de suas vidas; é bastante seguro supor que a maior parte desse trabalho não será feita em Java. Todo livro didático sério deve tentar prepará-los para algo mais fundamental do que a linguagem popular do momento.

Por outro lado, muitos detalhes de Java são essenciais para realizar programação prática. Neste livro, cobrimos as construções da linguagem Java no nível de detalhe necessário para ilustrar os conceitos em questão e implementar os trabalhos práticos. Algumas construções específicas de Java foram deliberadamente deixadas de fora da discussão.

Sabemos que alguns professores escolherão abordar determinados tópicos que não discutimos em detalhes. No entanto, em vez de tentar cobrir todos os assuntos possíveis - o que faria este livro crescer para cerca de 1.500 páginas -, lidamos com isso por meio de "ganchos". Esses ganchos são indicações, muitas vezes apresentadas na forma de perguntas, que levantam um assunto e fornecem referências para um apêndice ou material externo. Eles garantem que um tópico relevante seja mencionado em um momento apropriado e deixam ao leitor ou ao professor a decisão sobre o nível de detalhe em que ele será estudado. Assim, os ganchos funcionam como lembretes de que o assunto existe e como marcadores de um ponto da sequência em que uma discussão adicional pode ser inserida.

Cada professor pode decidir usar o livro exatamente como está, seguindo a sequência sugerida, ou explorar desvios e assuntos adicionais indicados pelos ganchos do texto.

Chapters also often include several questions suggesting discussion material related to the topic, but not discussed in this book. We fully expect teachers to discuss some of these questions in class, or students to research the answers as homework exercises.

Project-driven approach

The introduction of material in the book is project driven. The book discusses numerous programming projects and provides many exercises. Instead of introducing a new construct and then providing an exercise to apply this construct to solve a task, we first provide a goal and a problem. Analyzing the problem at hand determines what kinds of solutions we need. As a consequence, language constructs are introduced as they are needed to solve the problems before us.

Early chapters provide at least two discussion examples. These are projects that are discussed in detail to illustrate the important concepts of each chapter. Using two very different examples supports the iterative approach: each concept is revisited in a different context after it is introduced.

In designing this book we have tried to use a lot of different example projects. This will hopefully serve to capture the reader’s interest, and also illustrate the variety of different contexts in which the concepts can be applied. We hope that our projects serve to give teachers good starting points and many ideas for a wide variety of interesting assignments.

The implementation for all our projects is written very carefully, so that many peripheral issues may be studied by reading the projects’ source code. We are strong believers in learning by reading and imitating good examples. For this to work, however, it’s important that the examples are well written and worth imitating. We have tried to create great examples.

All projects are designed as open-ended problems. While one or more versions of each problem are discussed in detail in the book, the projects are designed so that further extensions and improvements can be done as student projects. Complete source code for all projects is included. A list of projects discussed in this book is provided on page xxv.

Concept sequence rather than language constructs

One other aspect that distinguishes this book from many others is that it is structured along fundamental software development tasks, not necessarily according to the particular Java language constructs. One indicator of this is the chapter headings. In this book you will not find traditional chapter titles such as “Primitive data types” or “Control structures.” Structuring by fundamental development tasks allows us to present a more general introduction that is not driven by intricacies of the particular programming language utilized. We also believe that it is easier for students to follow the motivation of the introduction, and that it makes much more interesting reading.

As a result of this approach, it is less straightforward to use the book as a reference book. Introductory textbooks and reference books have different, partly competing, goals. To a

PREFÁCIO

Os capítulos também incluem com frequência várias perguntas que sugerem material de discussão relacionado ao assunto, mas que não é desenvolvido no próprio livro. Esperamos que os professores discutam algumas dessas questões em sala de aula ou que os estudantes pesquisem as respostas como exercícios para casa.

Abordagem orientada por projetos

A apresentação do material neste livro é orientada por projetos. A obra discute numerosos projetos de programação e fornece muitos exercícios. Em vez de apresentar uma nova construção da linguagem e depois oferecer um exercício para aplicá-la na solução de uma tarefa, começamos apresentando um objetivo e um problema. A análise do problema determina os tipos de solução de que precisamos. Como consequência, as construções da linguagem são introduzidas no momento em que se tornam necessárias para resolver os problemas que estão diante de nós.

Os capítulos iniciais fornecem pelo menos dois exemplos para discussão. São projetos analisados em detalhes para ilustrar os conceitos importantes de cada capítulo. O uso de dois exemplos bastante diferentes apoia a abordagem iterativa: cada conceito é revisitado em um contexto diferente depois de ser apresentado.

Ao projetar este livro, procuramos utilizar muitos projetos de exemplo diferentes. Esperamos que isso ajude a despertar o interesse do leitor e também a demonstrar a variedade de contextos nos quais os conceitos podem ser aplicados. Esperamos ainda que nossos projetos ofereçam aos professores bons pontos de partida e muitas ideias para uma ampla variedade de atividades interessantes.

A implementação de todos os nossos projetos foi escrita com muito cuidado, de modo que diversas questões periféricas possam ser estudadas pela leitura do código-fonte. Acreditamos firmemente na aprendizagem por meio da leitura e da imitação de bons exemplos. Para que isso funcione, entretanto, é importante que os exemplos sejam bem escritos e dignos de serem imitados. Procuramos criar excelentes exemplos.

Todos os projetos foram concebidos como problemas abertos. Embora uma ou mais versões de cada problema sejam discutidas em detalhes no livro, os projetos foram preparados para permitir extensões e melhorias adicionais como trabalhos dos estudantes. O código-fonte completo de todos os projetos está incluído. Uma lista dos projetos discutidos neste livro é apresentada na página xxv.

Sequência de conceitos em vez de construções da linguagem

Outro aspecto que distingue este livro de muitos outros é que ele está estruturado em torno de tarefas fundamentais de desenvolvimento de software, e não necessariamente de acordo com construções específicas da linguagem Java. Um indicador disso são os títulos dos capítulos. Você não encontrará aqui títulos tradicionais como "Tipos de dados primitivos" ou "Estruturas de controle". Organizar o livro pelas tarefas fundamentais de desenvolvimento permite apresentar uma introdução mais geral, que não é conduzida pelas particularidades da linguagem de programação utilizada. Também acreditamos que dessa forma é mais fácil para os estudantes acompanhar a motivação da apresentação e que a leitura se torna muito mais interessante.

Como consequência dessa abordagem, o uso do livro como obra de referência é menos direto. Livros introdutórios e livros de referência possuem objetivos diferentes e, em parte, concorrentes.

certain extent a book can try to be both, but compromises have to be made at certain points. Our book is clearly designed as a textbook, and wherever a conflict occurred, the textbook style took precedence over its use as a reference book.

We have, however, provided support for use as a reference book by listing the Java constructs introduced in each chapter in the chapter introduction.

Chapter sequence

Chapter 1 deals with the most fundamental concepts of object orientation: objects, classes, and methods. It gives a solid, hands-on introduction to these concepts without going into the details of Java syntax. We briefly introduce the concept of abstraction. This will be a thread that runs through many chapters. Chapter 1 also gives a first look at some source code. We do this by using an example of graphical shapes that can be interactively drawn, and a second example of a simple laboratory class enrollment system.

Chapter 2 opens up class definitions and investigates how Java source code is written to create behavior of objects. We discuss how to define fields and implement methods, and point out the crucial role of the constructor in setting up an object’s state as embodied in its fields. Here, we also introduce the first types of statement. The main example is an implementation of a ticket machine. We also investigate the laboratory class example from Chapter 1 a bit further.

Chapter 3 then enlarges the picture to discuss interaction of multiple objects. We see how objects can collaborate by invoking each other’s methods to perform a common task. We also discuss how one object can create other objects. A digital alarm clock display is discussed that uses two number display objects to show hours and minutes. A version of the project that includes a GUI picks up on a running theme of the book—that we often provide additional code for the interested and able student to explore, without covering it in detail in the text. As a second major example, we examine a simulation of an email system in which messages can be sent between mail clients.

In Chapter 4 we continue by building more extensive structures of objects and pick up again on the themes of abstraction and object interaction from the preceding chapters. Most importantly, we start using collections of objects. We implement an organizer for music files and an auction system to introduce collections. At the same time, we discuss iteration over collections, and have a first look at the *for-each* and *while* loops. The first collection being used is an `ArrayList`.

Chapter 5 presents the first advanced section (a section that can be skipped if time is short): It is an introduction to functional programming constructs. The functional constructs present an alternative to the imperative collection processing discussed in Chapter 4. The same problems can be solved without these techniques, but functional constructs open some more elegant ways to achieve our goals. This chapter gives an introduction to the functional approach in general, and introduces a few of Java’s language constructs.

Chapter 6 deals with libraries and interfaces. We introduce the Java library and discuss some important library classes. More importantly, we explain how to read and understand the library documentation. The importance of writing documentation in software development

PREFÁCIO

Até certo ponto, um livro pode tentar cumprir as duas funções, mas alguns compromissos precisam ser assumidos. Nossa obra foi claramente concebida como um livro didático e, sempre que surgiu um conflito, o estilo de livro didático teve prioridade sobre seu uso como referência.

Ainda assim, oferecemos algum suporte ao uso como referência ao listar, na introdução de cada capítulo, as construções de Java apresentadas nele.

Sequência dos capítulos

O Capítulo 1 trata dos conceitos mais fundamentais da orientação a objetos: objetos, classes e métodos. Ele oferece uma introdução sólida e prática a esses conceitos sem entrar nos detalhes da sintaxe Java. Apresentamos brevemente o conceito de abstração, que será um tema recorrente em muitos capítulos. O Capítulo 1 também apresenta um primeiro contato com código-fonte. Fazemos isso usando um exemplo de formas gráficas que podem ser desenhadas interativamente e um segundo exemplo de um sistema simples de matrícula de estudantes em turmas de laboratório.

O Capítulo 2 abre as definições de classes e investiga como o código-fonte Java é escrito para criar o comportamento dos objetos. Discutimos como definir campos e implementar métodos e destacamos o papel crucial do construtor na configuração do estado de um objeto, representado por seus campos. Aqui também apresentamos os primeiros tipos de instrução. O exemplo principal é a implementação de uma máquina de bilhetes. Também investigamos um pouco mais o exemplo das turmas de laboratório do Capítulo 1.

O Capítulo 3 amplia o quadro para discutir a interação entre vários objetos. Vemos como os objetos podem colaborar chamando os métodos uns dos outros para executar uma tarefa comum. Também discutimos como um objeto pode criar outros objetos. É apresentado um mostrador de relógio digital que usa dois objetos de exibição numérica para mostrar horas e minutos. Uma versão do projeto que inclui uma GUI retoma um tema recorrente do livro: frequentemente fornecemos código adicional para que estudantes interessados e preparados possam explorá-lo, sem necessariamente discuti-lo em detalhes no texto. Como segundo grande exemplo, examinamos a simulação de um sistema de e-mail no qual mensagens podem ser enviadas entre clientes de correio eletrônico.

No Capítulo 4, continuamos construindo estruturas de objetos mais amplas e retomamos os temas de abstração e interação entre objetos dos capítulos anteriores. Mais importante ainda, começamos a utilizar coleções de objetos. Implementamos um organizador de arquivos de música e um sistema de leilões para introduzir coleções. Ao mesmo tempo, discutimos iteração sobre coleções e temos um primeiro contato com os laços *for-each* e *while*. A primeira coleção utilizada é um `ArrayList`.

O Capítulo 5 apresenta a primeira seção avançada - uma seção que pode ser pulada se houver pouco tempo. Trata-se de uma introdução às construções de programação funcional. Essas construções apresentam uma alternativa ao processamento imperativo de coleções discutido no Capítulo 4. Os mesmos problemas podem ser resolvidos sem essas técnicas, mas as construções funcionais oferecem maneiras mais elegantes de alcançar alguns objetivos. O capítulo apresenta a abordagem funcional em geral e algumas das construções correspondentes da linguagem Java.

O Capítulo 6 trata de bibliotecas e interfaces. Introduzimos a biblioteca Java e discutimos algumas classes importantes. Mais importante ainda, explicamos como ler e compreender a documentação da biblioteca. Também discutimos a importância de escrever documentação em projetos de desenvolvimento de software.

projects is discussed, and we end by practicing how to write suitable documentation for our own classes. Random, Set, and Map are examples of classes that we encounter in this chapter. We implement an *Eliza*-like dialog system and a graphical simulation of a bouncing ball to apply these classes.

Chapter 7 concentrates on one specific—but very special—type of collection: arrays. Arrays processing and the associated types of loops are discussed in detail.

In Chapter 8 we discuss more formally the issues of dividing a problem domain into classes for implementation. We introduce issues of good class design, including concepts such as responsibility-driven design, coupling, cohesion, and refactoring. An interactive, text-based adventure game (*World of Zuul*) is used for this discussion. We go through several iterations of improving the internal class structure of the game and extending its functionality, and end with a long list of proposals for extensions that may be done as student projects.

Chapter 9 deals with a whole group of issues connected to producing correct, understandable, and maintainable classes. It covers issues ranging from writing clear code (including style and commenting) to testing and debugging. Test strategies are introduced, including formalized regression testing using JUnit, and a number of debugging methods are discussed in detail. We use an example of an online shop and an implementation of an electronic calculator to discuss these topics.

Chapters 10 and 11 introduce inheritance and polymorphism, with many of the related detailed issues. We discuss a part of a social network to illustrate the concepts. Issues of code inheritance, subtyping, polymorphic method calls, and overriding are discussed in detail.

In Chapter 12 we implement a predator/prey simulation. This serves to discuss additional abstraction mechanisms based on inheritance, namely interfaces and abstract classes.

Chapter 13 develops an image viewer and a graphical user interface for the music organizer (first encountered in Chapter 4). Both examples serve to discuss how to build graphical user interfaces (GUIs).

Chapter 14 then picks up the difficult issue of how to deal with errors. Several possible problems and solutions are discussed, and Java’s exception-handling mechanism is discussed in detail. We extend and improve an address book application to illustrate the concepts. Input/output is used as a case study where error-handling is an essential requirement.

Chapter 15 discusses in more detail the next level of abstraction: How to structure a vaguely described problem into classes and methods. In previous chapters we have assumed that large parts of the application structure already exist, and we have made improvements. Now it is time to discuss how we can get started from a clean slate. This involves detailed discussion of what the classes should be that implement our application, how they interact, and how responsibilities should be distributed. We use Class/Responsibilities/Collaborators (CRC) cards to approach this problem, while designing a cinema booking system.

In Chapter 16 we to bring everything together and integrate topics from the previous chapters of the book. It is a complete case study, starting with the application design, through design of the class interfaces, down to discussing many important functional and non-functional characteristics and implementation details. Concepts discussed in earlier chapters (such as reliability, data structures, class design, testing, and extendibility) are applied again in a new context.

PREFÁCIO

Ao final do Capítulo 6, praticamos como escrever documentação apropriada para nossas próprias classes. Random, Set e Map são exemplos de classes encontradas nesse capítulo. Implementamos um sistema de diálogo semelhante ao Eliza e uma simulação gráfica de uma bola quicando para aplicar essas classes.

O Capítulo 7 concentra-se em um tipo específico - e muito especial - de coleção: arrays. O processamento de arrays e os tipos de laço associados são discutidos em detalhes.

No Capítulo 8, discutimos de maneira mais formal as questões envolvidas na divisão de um domínio de problema em classes para implementação. Apresentamos aspectos de um bom projeto de classes, incluindo conceitos como projeto orientado a responsabilidades, acoplamento, coesão e refatoração. Um jogo de aventura interativo baseado em texto, World of Zuul, é utilizado nessa discussão. Passamos por várias iterações de melhoria da estrutura interna de classes do jogo e de extensão de suas funcionalidades e terminamos com uma longa lista de propostas de extensões que podem ser desenvolvidas como projetos dos estudantes.

O Capítulo 9 trata de todo um conjunto de questões relacionadas à produção de classes corretas, compreensíveis e fáceis de manter. Ele aborda temas que vão desde a escrita de código claro - incluindo estilo e comentários - até testes e depuração. São apresentadas estratégias de teste, incluindo testes de regressão formalizados com JUnit, e diversos métodos de depuração são discutidos em detalhes. Usamos como exemplos uma loja on-line e a implementação de uma calculadora eletrônica.

Os Capítulos 10 e 11 apresentam herança e polimorfismo, juntamente com muitos dos detalhes relacionados. Discutimos parte de uma rede social para ilustrar esses conceitos. Herança de código, subtipagem, chamadas polimórficas de métodos e sobrescrita são estudadas em detalhes.

No Capítulo 12, implementamos uma simulação de predador e presa. Ela serve para discutir mecanismos adicionais de abstração baseados em herança, especificamente interfaces e classes abstratas.

O Capítulo 13 desenvolve um visualizador de imagens e uma interface gráfica para o organizador de músicas, apresentado pela primeira vez no Capítulo 4. Os dois exemplos são usados para discutir como construir interfaces gráficas com o usuário (GUIs).

O Capítulo 14 aborda então a difícil questão de como lidar com erros. Vários problemas possíveis e respectivas soluções são discutidos, assim como o mecanismo de tratamento de exceções do Java, em detalhes. Estendemos e melhoramos uma aplicação de catálogo de endereços para ilustrar os conceitos. Entrada e saída são utilizadas como estudo de caso em que o tratamento de erros é um requisito essencial.

O Capítulo 15 discute com mais detalhes o próximo nível de abstração: como estruturar em classes e métodos um problema descrito de forma vaga. Nos capítulos anteriores, partimos da suposição de que grandes partes da estrutura da aplicação já existiam e fizemos melhorias. Agora é hora de discutir como começar a partir de uma folha em branco. Isso envolve uma discussão detalhada sobre quais classes devem implementar a aplicação, como interação e como as responsabilidades devem ser distribuídas. Usamos cartões Class/Responsibilities/Collaborators (CRC) para abordar esse problema durante o projeto de um sistema de reserva de ingressos de cinema.

No Capítulo 16, reunimos tudo e integramos tópicos dos capítulos anteriores. É um estudo de caso completo que começa pelo projeto da aplicação, passa pelo projeto das interfaces das classes e chega à discussão de várias características funcionais e não funcionais importantes, além de detalhes de implementação. Conceitos discutidos anteriormente - como confiabilidade, estruturas de dados, projeto de classes, testes e extensibilidade - são aplicados novamente em um novo contexto.

Supplements

VideoNotes: VideoNotes are Pearson’s visual tool designed to teach students key programming concepts and techniques. These short step-by-step videos demonstrate how to solve problems from design through coding. VideoNotes allow for self-paced instruction with easy navigation including the ability to select, play, rewind, fast-forward, and stop within each VideoNote exercise.

VideoNotes are located at www.pearsonglobaleditions.com/Barnes. Twelve months of prepaid access are included with the purchase of a new textbook.

Book website: All projects used as discussion examples and exercises in this book are available for download on the book’s website, at <http://www.bluej.org/objects-first/>. The website also provides links to download BlueJ, and other resources.

Companion website for students: The following resources are available to all readers of this book at its Companion Website, located at www.pearsonglobaleditions.com/Barnes:

- Program style guide for all examples in the book
- Links to further material of interest
- Complete source code for all projects

Instructor resources: The following supplements are available to qualified instructors only:

- Solutions to end-of-chapter exercises
- PowerPoint slides

Visit the Pearson Instructor Resource Center at www.pearsonglobaleditions.com/Barnes to register for access or contact your local Pearson representative.

The Blueroom

Perhaps more important than the static web site resources is a very active community forum that exists for instructors who teach with BlueJ and this book. It is called the *Blueroom* and can be found at

<http://blueroom.bluej.org>

The Blueroom contains a resource collection with many teaching resources shared by other teachers, as well as a discussion forum where instructors can ask questions, discuss issues, and stay up-to-date with the latest developments. Many other teachers, as well as developers of BlueJ and the authors of this book, can be contacted in the Blueroom.

PREFÁCIO

Suplementos

VideoNotes: VideoNotes são a ferramenta visual da Pearson criada para ensinar aos estudantes conceitos e técnicas fundamentais de programação. Esses vídeos curtos, apresentados passo a passo, demonstram como resolver problemas desde o projeto até a codificação. Os VideoNotes permitem estudo no ritmo do próprio aluno, com navegação simples e a possibilidade de selecionar, reproduzir, retroceder, avançar rapidamente e parar em cada exercício.

Os VideoNotes estão disponíveis em www.pearsonglobaleditions.com/Barnes. A compra de um livro novo inclui doze meses de acesso pré-pago.

Site do livro: todos os projetos utilizados como exemplos de discussão e exercícios estão disponíveis para download no site do livro, em <http://www.bluej.org/objects-first/>. O site também fornece links para baixar o BlueJ e outros recursos.

Site complementar para estudantes: os seguintes recursos estão disponíveis para todos os leitores no Companion Website, em www.pearsonglobaleditions.com/Barnes:

- guia de estilo de programação para todos os exemplos do livro;
- links para outros materiais de interesse;
- código-fonte completo de todos os projetos.

Recursos para professores: os seguintes suplementos estão disponíveis apenas para professores qualificados:

- soluções dos exercícios de final de capítulo;
- slides em PowerPoint.

Visite o Pearson Instructor Resource Center em www.pearsonglobaleditions.com/Barnes para registrar-se e solicitar acesso, ou entre em contato com o representante local da Pearson.

The Blueroom

Talvez ainda mais importante do que os recursos estáticos dos sites seja um fórum comunitário muito ativo para professores que ensinam utilizando o BlueJ e este livro. Ele se chama Blueroom e pode ser encontrado em: <http://blueroom.bluej.org>

O Blueroom contém uma coleção de recursos didáticos compartilhados por outros professores e um fórum de discussão no qual professores podem fazer perguntas, discutir questões e acompanhar os desenvolvimentos mais recentes. Muitos outros professores, bem como desenvolvedores do BlueJ e os autores deste livro, podem ser contatados no Blueroom.



List of Projects Discussed in Detail in This Book

Figures	(Chapter 1)
Simple drawing with some geometrical shapes; illustrates creation of objects, method calling, and parameters.	
House	(Chapter 1)
An example using shape objects to draw a picture; introduces source code, Java syntax, and compilation.	
Lab-classes	(Chapter 1, 2, 10)
A simple example with classes of students; illustrates objects, fields, and methods. Used again in Chapter 10 to add inheritance.	
Ticket-machine	(Chapter 2)
A simulation of a ticket vending machine for train tickets; introduces more about fields, constructors, accessor and mutator methods, parameters, and some simple statements.	
Book-exercise	(Chapter 2)
Storing details of a book. Reinforcing the constructs used in the ticket-machine example.	
Clock-display	(Chapter 3)
An implementation of a display for a digital clock; illustrates the concepts of abstraction, modularization, and object interaction. Includes a version with an animated GUI.	
Mail system	(Chapter 3)
A simple simulation of an email system. Used to demonstrate object creation and interaction.	
Music-organizer	(Chapter 4, 11)
An implementation of an organizer for music tracks; used to introduce collections and loops. Includes the ability to play MP3 files. A GUI is added in Chapter 11.	
Auction	(Chapter 4)
An auction system. More about collections and loops, this time with iterators.	

Tradução PT-BR - página original 26

LISTA DE PROJETOS DISCUTIDOS EM DETALHES NESTE LIVRO

Figures (Capítulo 1)
Desenho simples com algumas formas geométricas; ilustra a criação de objetos, chamadas de métodos e parâmetros.

House (Capítulo 1)
Exemplo que utiliza objetos de formas para desenhar uma figura; apresenta código-fonte, sintaxe Java e compilação.

Lab-classes (Capítulos 1, 2 e 10)
Exemplo simples com classes de estudantes; ilustra objetos, campos e métodos. É utilizado novamente no Capítulo 10 para acrescentar herança.

Ticket-machine (Capítulo 2)
Simulação de uma máquina de venda de bilhetes de trem; apresenta mais detalhes sobre campos, construtores, métodos acessores e modificadores, parâmetros e algumas instruções simples.

Book-exercise (Capítulo 2)
Armazenamento de informações de um livro. Reforça as construções utilizadas no exemplo ticket-machine.

Clock-display (Capítulo 3)
Implementação do mostrador de um relógio digital; ilustra os conceitos de abstração, modularização e interação entre objetos. Inclui uma versão com GUI animada.

Mail system (Capítulo 3)
Simulação simples de um sistema de e-mail. Utilizada para demonstrar criação e interação de objetos.

Music-organizer (Capítulos 4 e 11)
Implementação de um organizador de faixas de música; utilizada para introduzir coleções e laços. Inclui a capacidade de reproduzir arquivos MP3. Uma GUI é acrescentada no Capítulo 11.

Auction (Capítulo 4)
Sistema de leilões. Apresenta mais sobre coleções e laços, desta vez utilizando iteradores.

Animal-monitoring (Chapter 5, 6)

A system to monitor animal populations, e.g., in a national park. This is used to introduce functional processing of collections.

Tech-support (Chapter 6, 14)

An implementation of an *Eliza*-like dialog program used to provide “technical support” to customers; introduces use of library classes in general and some specific classes in particular; reading and writing of documentation.

Scribble (Chapter 6)

A shape-drawing program to support learning about classes from their interfaces.

Bouncing-balls (Chapter 6)

A graphical animation of bouncing balls; demonstrates interface/implementation separation and simple graphics.

Weblog-analyzer (Chapter 7, 14)

A program to analyze web access log files; introduces arrays and for loops.

Automaton (Chapter 7)

A series of examples of a cellular automaton. Used to gain practice with array programming.

Brain (Chapter 7)

A version of *Brian’s Brain*, which we use to discuss two-dimensional arrays.

World-of-zuul (Chapter 8, 11, 14)

A text-based, interactive adventure game. Highly extendable, makes a great open-ended student project. Used here to discuss good class design, coupling, and cohesion. Used again in Chapter 11 as an example for use of inheritance.

Online-shop (Chapter 9)

The early stages of an implementation of a part of an online shopping website, dealing with user comments; used to discuss testing and debugging strategies.

Calculator (Chapter 9)

An implementation of a desk calculator. This example reinforces concepts introduced earlier, and is used to discuss testing and debugging.

Bricks (Chapter 9)

A simple debugging exercise; models filling pallets with bricks for simple computations.

Network (Chapter 10, 11)

Part of a social network application. This project is discussed and then extended in great detail to introduce the foundations of inheritance and polymorphism.

LISTA DE PROJETOS DISCUTIDOS EM DETALHES NESTE LIVRO

Animal-monitoring (Capítulos 5 e 6)
Sistema para monitorar populações de animais, por exemplo em um parque nacional. É utilizado para apresentar o processamento funcional de coleções.

Tech-support (Capítulos 6 e 14)
Implementação de um programa de diálogo semelhante ao Eliza usado para fornecer "suporte técnico" a clientes; apresenta o uso de classes de biblioteca em geral e de algumas classes específicas em particular, além da leitura e escrita de documentação.

Scribble (Capítulo 6)
Programa para desenhar formas, usado para apoiar a aprendizagem sobre classes a partir de suas interfaces.

Bouncing-balls (Capítulo 6)
Animação gráfica de bolas quicando; demonstra a separação entre interface e implementação e gráficos simples.

Weblog-analyzer (Capítulos 7 e 14)
Programa para analisar arquivos de log de acesso à web; apresenta arrays e laços for.

Automaton (Capítulo 7)
Série de exemplos de um autômato celular. Utilizada para praticar programação com arrays.

Brain (Capítulo 7)
Uma versão de Brian's Brain, utilizada para discutir arrays bidimensionais.

World-of-zuul (Capítulos 8, 11 e 14)
Jogo de aventura interativo baseado em texto. É altamente extensível e constitui um excelente projeto aberto para estudantes. É utilizado aqui para discutir bom projeto de classes, acoplamento e coesão e retorna no Capítulo 11 como exemplo de uso de herança.

Online-shop (Capítulo 9)
Estágios iniciais da implementação de parte de um site de comércio eletrônico, tratando de comentários de usuários; usado para discutir estratégias de teste e depuração.

Calculator (Capítulo 9)
Implementação de uma calculadora de mesa. O exemplo reforça conceitos apresentados anteriormente e é utilizado para discutir testes e depuração.

Bricks (Capítulo 9)
Exercício simples de depuração; modela o preenchimento de paletes com tijolos para realizar cálculos simples.

Network (Capítulos 10 e 11)
Parte de uma aplicação de rede social. O projeto é discutido e depois ampliado em muitos detalhes para apresentar os fundamentos de herança e polimorfismo.

Foxes-and-rabbits	(Chapter 12)
A classic predator–prey simulation; reinforces inheritance concepts and adds abstract classes and interfaces.	
Image-viewer	(Chapter 13)
A simple image-viewing and -manipulation application. We concentrate mainly on building the GUI.	
Music-player	(Chapter 13)
A GUI is added to the <i>music-organizer</i> project of Chapter 4 as another example of building GUIs.	
Address-book	(Chapter 14)
An implementation of an address book with an optional GUI interface. Lookup is flexible: entries can be searched by partial definition of name or phone number. This project makes extensive use of exceptions.	
Cinema-booking-system	(Chapter 15)
A system to manage advance seat bookings in a cinema. This example is used in a discussion of class discovery and application design. No code is provided, as the example represents the development of an application from a blank sheet of paper.	
Taxi-company	(Chapter 16)
The taxi example is a combination of a booking system, management system, and simulation. It is used as a case study to bring together many of the concepts and techniques discussed throughout the book.	

Tradução PT-BR - página original 28

LISTA DE PROJETOS DISCUTIDOS EM DETALHES NESTE LIVRO

Foxes-and-rabbits (Capítulo 12)
Simulação clássica de predador e presa; reforça conceitos de herança e acrescenta classes abstratas e interfaces.
Image-viewer (Capítulo 13)
Aplicação simples para visualizar e manipular imagens. A discussão se concentra principalmente na construção da GUI.
Music-player (Capítulo 13)
Uma GUI é acrescentada ao projeto music-organizer do Capítulo 4 como outro exemplo de construção de interfaces gráficas.
Address-book (Capítulo 14)
Implementação de um catálogo de endereços com uma interface GUI opcional. A busca é flexível: as entradas podem ser pesquisadas por parte do nome ou do número de telefone. O projeto faz uso extensivo de exceções.
Cinema-booking-system (Capítulo 15)
Sistema para gerenciar reservas antecipadas de assentos em um cinema. O exemplo é usado em uma discussão sobre descoberta de classes e projeto de aplicações. Nenhum código é fornecido, pois o exemplo representa o desenvolvimento de uma aplicação começando a partir de uma folha em branco.
Taxi-company (Capítulo 16)
O exemplo da empresa de táxis combina um sistema de reservas, um sistema de gerenciamento e uma simulação. É utilizado como estudo de caso para reunir muitos dos conceitos e técnicas discutidos ao longo do livro.

AGRADECIMENTOS

Muitas pessoas contribuíram, de diversas maneiras, para este livro e tornaram possível sua criação.

Em primeiro lugar, e mais importante, devemos mencionar John Rosenberg. É apenas por uma coincidência das circunstâncias que John não figura entre os autores deste livro. Desde o início, ele foi uma das forças impulsionadoras do desenvolvimento do BlueJ, das ideias e da pedagogia que o sustentam, e durante vários anos conversamos sobre a escrita deste livro. Grande parte do material presente aqui foi desenvolvida em discussões com John. O simples fato de o dia possuir apenas vinte e quatro horas - muitas das quais já eram consumidas por inúmeras outras tarefas - impediu que ele participasse diretamente da escrita. John contribuiu significativamente para a versão original deste texto e ajudou a melhorá-lo de muitas maneiras. Valorizamos imensamente sua amizade e colaboração.

Várias outras pessoas ajudaram a tornar o BlueJ o que ele é: Bruce Quig, Davin McCall e Andrew Patterson, na Austrália, e Ian Utting, Poul Henriksen e Neil Brown, na Inglaterra. Todos trabalharam no BlueJ durante muitos anos, melhorando e ampliando o projeto e a implementação paralelamente às demais responsabilidades profissionais. Sem esse trabalho, o BlueJ jamais teria alcançado a qualidade e a popularidade que possui atualmente, e talvez este livro nunca tivesse sido escrito.

Outra contribuição importante que tornou possível a criação do BlueJ e deste livro foi o apoio extremamente generoso inicialmente da Sun Microsystems e, agora, da Oracle. A Sun apoiou o BlueJ generosamente durante muitos anos e, quando a Oracle adquiriu a Sun, esse apoio continuou. Somos profundamente gratos por essa contribuição essencial.

A equipe da Pearson também fez um excelente trabalho para tornar este livro realidade, conseguindo colocá-lo no mundo e evitando o pior medo de todo autor: que sua obra passe despercebida. Agradecimentos especiais são devidos à nossa editora, Tracy Johnson, e a Camille Trentacoste e Carole Snyder, que nos apoiaram durante todo o processo de escrita e produção.

David gostaria de acrescentar seus agradecimentos pessoais aos funcionários e estudantes da School of Computing da Universidade de Kent. Sempre foi um privilégio ensinar os estudantes que frequentaram seu curso introdutório de orientação a objetos. Eles também fornecem o estímulo e a motivação essenciais que tornam o ensino tão prazeroso. Sem a valiosa ajuda de supervisores de pós-graduação experientes e prestativos, seria impossível conduzir as aulas.

Michael gostaria de agradecer a Davin e Neil pelo excelente trabalho na construção e manutenção do BlueJ e de nossos sites comunitários. Sem uma equipe tão boa, nada disso funcionaria.

Acknowledgments

Many people have contributed in many different ways to this book and made its creation possible.

First, and most importantly, John Rosenberg must be mentioned. It is only by coincidence of circumstance that John is not one of the authors of this book. He was one of the driving forces in the development of BlueJ and the ideas and pedagogy behind it from the very beginning, and we talked about the writing of this book for several years. Much of the material in this book was developed in discussions with John. Simply the fact that there are only twenty-four hours in a day, too many of which were already taken up with too much other work, prevented him from actually writing this book. John contributed significantly to the original version of this text and helped improve it in many ways. We have appreciated his friendship and collaboration immensely.

Several other people have helped make BlueJ what it is: Bruce Quig, Davin McCall, and Andrew Patterson in Australia, and Ian Utting, Poul Henriksen, and Neil Brown in England. All have worked on BlueJ for many years, improving and extending the design and implementation in addition to their other work commitments. Without their work, BlueJ would never have reached the quality and popularity it has today, and this book might never have been written.

Another important contribution that made the creation of BlueJ and this book possible was very generous support first from Sun Microsystems and now from Oracle. Sun has very generously supported BlueJ for many years, and when Oracle acquired Sun this support has continued. We are very grateful for this crucial contribution.

The Pearson team also has done a terrific job in making this book happen, managing to bring it into the world and avert every author’s worst fear—that his book might go unnoticed. Particular thanks are due to our editor, Tracy Johnson, and to Camille Trentacoste and Carole Snyder, who supported us through the writing and production process.

David would like to add his personal thanks to both staff and students of the School of Computing at the University of Kent. The students who have taken his introductory OO course have always been a privilege to teach. They also provide the essential stimulus and motivation that makes teaching so much fun. Without the valuable assistance of knowledgeable and supportive postgraduate supervisors, running classes would be impossible.

Michael would like to thank Davin and Neil who have done such excellent work in building and maintaining BlueJ and our community web sites. Without such a great team none of this could work.

PARTE 1
FUNDAMENTOS DA ORIENTAÇÃO A OBJETOS

Capítulo 1 - Objetos e Classes
Capítulo 2 - Compreendendo Definições de Classes
Capítulo 3 - Interação entre Objetos
Capítulo 4 - Agrupando Objetos
Capítulo 5 - Processamento Funcional de Coleções (Avançado)
Capítulo 6 - Comportamento Mais Sofisticado
Capítulo 7 - Coleções de Tamanho Fixo - Arrays
Capítulo 8 - Projetando Classes
Capítulo 9 - Objetos Bem Comportados

- Chapter 1** **Objects and Classes**
- Chapter 2** **Understanding Class Definitions**
- Chapter 3** **Object Interaction**
- Chapter 4** **Grouping Objects**
- Chapter 5** **Functional Processing of Collections (Advanced)**
- Chapter 6** **More-Sophisticated Behavior**
- Chapter 7** **Fixed-Size Collections—Arrays**
- Chapter 8** **Designing Classes**
- Chapter 9** **Well-Behaved Objects**

Esta página foi deixada intencionalmente em branco.

This page intentionally left blank

CHAPTER

1

Objects and Classes

Main concepts discussed in this chapter:

- objects
- methods
- classes
- parameters

It's time to jump in and get started with our discussion of object-oriented programming. Learning to program requires a mix of some theory and a lot of practice. In this book, we will present both, so that the two reinforce each other.

At the heart of object orientation are two concepts that we have to understand first: *objects* and *classes*. These concepts form the basis of all programming in object-oriented languages. So let us start with a brief discussion of these two foundations.

1.1 Objects and classes

Concept

Java **objects** model objects from a problem domain.

If you write a computer program in an object-oriented language, you are creating, in your computer, a model of some part of the world. The parts that the model is built up from are the *objects* that appear in the problem domain. These objects must be represented in the computer model being created. The objects from the problem domain vary with the program you are writing. They may be words and paragraphs if you are programming a word processor, users and messages if you are working on a social-network system, or monsters if you are writing a computer game.

Objects may be categorized. A class describes,—in an abstract way,—all objects of a particular kind.

We can make these abstract notions clearer by looking at an example. Assume you want to model a traffic simulation. One kind of entity you then have to deal with is cars. What is a car in our context? Is it a class or an object? A few questions may help us to make a decision.

What color is a car? How fast can it go? Where is it right now?

CAPÍTULO 1 - OBJETOS E CLASSES

Chegou a hora de começar nossa discussão sobre programação orientada a objetos. Aprender a programar exige uma combinação de teoria e muita prática. Neste livro, apresentaremos as duas coisas para que uma reforce a outra.

No centro da orientação a objetos estão dois conceitos que precisamos compreender primeiro: objetos e classes. Esses conceitos formam a base de toda programação em linguagens orientadas a objetos. Portanto, começaremos com uma breve discussão sobre esses dois fundamentos.

1.1 Objetos e classes

Quando você escreve um programa de computador em uma linguagem orientada a objetos, está criando, dentro do computador, um modelo de alguma parte do mundo. As partes que compõem esse modelo são os objetos existentes no domínio do problema. Esses objetos precisam ser representados no modelo computacional que está sendo criado. Os objetos do domínio do problema variam de acordo com o programa que você está escrevendo. Eles podem ser palavras e parágrafos se você estiver programando um processador de texto, usuários e mensagens se estiver trabalhando em um sistema de rede social, ou monstros se estiver escrevendo um jogo de computador.

Os objetos podem ser categorizados. Uma classe descreve, de maneira abstrata, todos os objetos de um determinado tipo.

Podemos tornar essas ideias abstratas mais claras com um exemplo. Suponha que você queira modelar uma simulação de trânsito. Um tipo de entidade com que precisará lidar são os carros. O que é um carro nesse contexto? É uma classe ou um objeto? Algumas perguntas podem nos ajudar a decidir.

Qual é a cor de um carro? Quão rápido ele consegue andar? Onde ele está neste momento?

Principais conceitos discutidos neste capítulo:

- objetos
- métodos
- classes
- parâmetros

Conceito: objetos Java modelam objetos de um domínio de problema.

Concept

Objects are created from **classes**. The class describes the kind of object; the objects represent individual instances of the class.

You will notice that we cannot answer these questions until we talk about one specific car. The reason is that the word “car” in this context refers to the *class* car; we are talking about cars in general, not about one particular car.

If I speak of “my old car that is parked at home in my garage,” we can answer the questions above. That car is red, it doesn’t go very fast, and it is in my garage. Now I am talking about an object—about one particular example of a car.

We usually refer to a particular object as an *instance*. We shall use the term “instance” quite regularly from now on. “Instance” is roughly synonymous with “object”; we refer to objects as instances when we want to emphasize that they are of a particular class (such as “this object is an instance of the class car”).

Before we continue this rather theoretical discussion, let us look at an example.

1.2 Creating objects

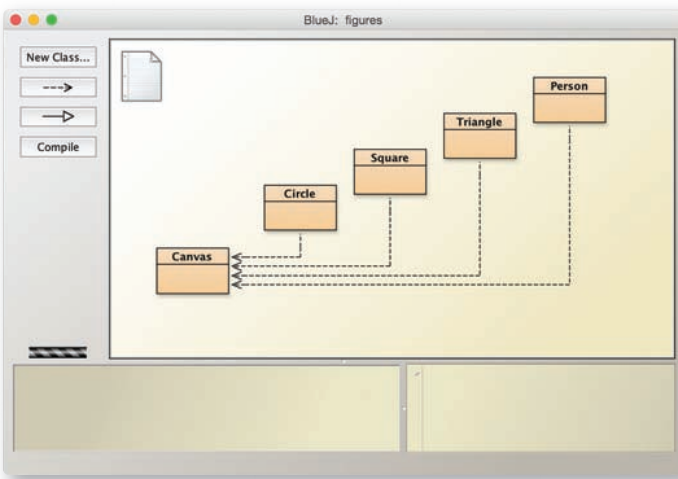
Start BlueJ and open the example named *figures*.¹ You should see a window similar to that shown in Figure 1.1.

In this window, a diagram should become visible. Every one of the colored rectangles in the diagram represents a class in our project. In this project, we have classes named **Circle**, **Square**, **Triangle**, **Person**, and **Canvas**.

Right-click on the **Circle** class and choose

`new Circle()`

Figure 1.1
The *figures* project in BlueJ



¹ We regularly expect you to undertake some activities and exercises while reading this book. At this point, we assume that you already know how to start BlueJ and open the example projects. If not, read Appendix A first.

CAPÍTULO 1 - OBJETOS E CLASSES

Você perceberá que não podemos responder às perguntas anteriores até falarmos de um carro específico. O motivo é que a palavra "carro", nesse contexto, refere-se à classe carro; estamos falando de carros em geral, e não de um carro em particular.

Se eu disser "meu carro velho que está estacionado em casa, na minha garagem", podemos responder às perguntas. Esse carro é vermelho, não anda muito rápido e está na minha garagem. Agora estou falando de um objeto - um exemplo específico de carro.

Normalmente nos referimos a um objeto específico como uma instância. Usaremos o termo "instância" regularmente daqui em diante. "Instância" é aproximadamente sinônimo de "objeto"; chamamos objetos de instâncias quando queremos enfatizar que eles pertencem a uma determinada classe, por exemplo: "este objeto é uma instância da classe carro".

Antes de continuar essa discussão bastante teórica, vejamos um exemplo.

1.2 Criando objetos

Inicie o BlueJ e abra o exemplo chamado *figures*. Você deverá ver uma janela semelhante à mostrada na Figura 1.1.

Nessa janela, deverá aparecer um diagrama. Cada um dos retângulos coloridos do diagrama representa uma classe do projeto. Nesse projeto, temos classes chamadas Circle, Square, Triangle, Person e Canvas.

Clique com o botão direito na classe Circle e escolha:

`new Circle()`

Nota: ao longo do livro, esperamos que você realize atividades e exercícios durante a leitura. Neste ponto, supomos que você já saiba iniciar o BlueJ e abrir os projetos de exemplo. Caso contrário, leia primeiro o Apêndice A.

Conceito: objetos são criados a partir de classes. A classe descreve o tipo de objeto; os objetos representam instâncias individuais da classe.

Figura 1.1 - O projeto *figures* no BlueJ.

Figure 1.2
An object on the
object bench



from the pop-up menu. The system asks you for a “name of the instance”; click OK—the default name supplied is good enough for now. You will see a red rectangle toward the bottom of the screen labeled “circle1” (Figure 1.2).

Convention We start names of classes with capital letters (such as `Circle`) and names of objects with lowercase letters (such as `circle1`). This helps to distinguish what we are talking about.

You have just created your first object! “Circle,” the rectangular icon in Figure 1.1, represents the class `Circle`; `circle1` is an object created from this class. The area at the bottom of the screen where the object is shown is called the *object bench*.

Exercise 1.1 Create another circle. Then create a square.

1.3 Calling methods

Right-click on one of the circle objects (not the class!), and you will see a pop-up menu with several operations (Figure 1.3). Choose `makeVisible` from the menu—this will draw a representation of this circle in a separate window (Figure 1.4).

Concept

We can communicate with objects by invoking **methods** on them. Objects usually do something if we invoke a method.

Exercise 1.2 What happens if you call `moveDown` twice? Or three times? What happens if you call `makeInvisible` twice?

The entries in the circle’s menu represent operations that you can use to manipulate the circle. These are called *methods* in Java. Using common terminology, we say that these

1.3 CHAMANDO MÉTODOS

No menu pop-up, escolha a opção indicada. O sistema solicitará um "nome da instância"; clique em OK - o nome padrão fornecido é suficiente por enquanto. Você verá um retângulo vermelho na parte inferior da tela, identificado como "circle1" (Figura 1.2).

Figura 1.2 - Um objeto na bancada de objetos.

Convenção: iniciamos nomes de classes com letras maiúsculas, como `Circle`, e nomes de objetos com letras minúsculas, como `circle1`. Isso ajuda a distinguir aquilo de que estamos falando.

Exercício 1.1 - Crie outro círculo. Depois, crie um quadrado.

Exercício 1.2 - O que acontece se você chamar `moveDown` duas vezes? E três vezes? O que acontece se chamar `makeInvisible` duas vezes?

1.3 Chamando métodos

Clique com o botão direito em um dos objetos círculo - e não na classe. Você verá um menu pop-up com diversas operações (Figura 1.3). Escolha `makeVisible` no menu; isso desenhará uma representação desse círculo em uma janela separada (Figura 1.4).

Você notará várias outras operações no menu do círculo. Tente chamar `moveRight` e `moveDown` algumas vezes para mover o círculo em direção ao canto da tela. Você também pode experimentar `makeInvisible` e `makeVisible` para ocultar e exibir o círculo.

Você acabou de criar seu primeiro objeto! "Circle", o ícone retangular da Figura 1.1, representa a classe `Circle`; `circle1` é um objeto criado a partir dessa classe. A área na parte inferior da tela em que o objeto é exibido é chamada de bancada de objetos (object bench).

As entradas do menu do círculo representam operações que podem ser usadas para manipular o círculo. Em Java, elas são chamadas de métodos. Usando a terminologia comum, dizemos que esses métodos são chamados ou invocados.

Conceito: podemos nos comunicar com objetos invocando métodos neles. Em geral, os objetos fazem alguma coisa quando invocamos um método.

Figure 1.3
An object's pop-up menu, listing its operations

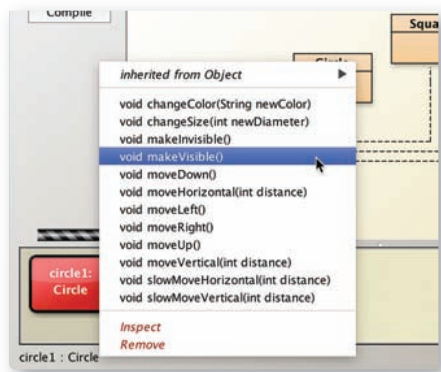
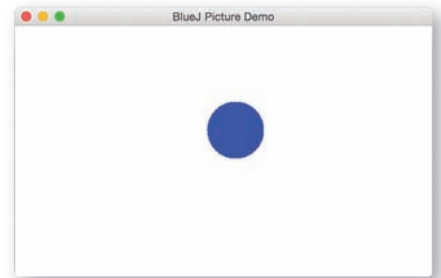


Figure 1.4
A drawing of a circle



methods are *called* or *invoked*. We shall use this proper terminology from now on. We might ask you to “invoke the **moveRight** method of **circle1**.”

1.4 Parameters

Concept
Methods can have **parameters** to provide additional information for a task.

Now invoke the **moveHorizontal** method. You will see a dialog appear that prompts you for some input (Figure 1.5). Type in 50 and click OK. You will see the circle move 50 pixels to the right.²

The **moveHorizontal** method that was just called is written in such a way that it requires some more information to execute. In this case, the information required is the distance—how far the circle should be moved. Thus, the **moveHorizontal** method is more flexible than the **moveRight** and **moveLeft** methods. The latter always move the circle a fixed distance, whereas **moveHorizontal** lets you specify how far you want to move the circle.

² A pixel is a single dot on your screen. Your screen is made up of a grid of single pixels.

CAPÍTULO 1 - OBJETOS E CLASSES

A partir de agora, usaremos essa terminologia adequada. Podemos, por exemplo, pedir que você "invoque o método **moveRight** de **circle1**".

1.4 Parâmetros

Agora invoque o método **moveHorizontal**. Uma caixa de diálogo aparecerá solicitando uma entrada (Figura 1.5). Digite 50 e clique em OK. Você verá o círculo se mover 50 pixels para a direita.

O método **moveHorizontal** que acabou de ser chamado foi escrito de modo a exigir uma informação adicional para ser executado. Neste caso, a informação necessária é a distância - o quanto o círculo deve ser deslocado. Assim, **moveHorizontal** é mais flexível do que **moveRight** e **moveLeft**. Estes últimos sempre deslocam o círculo por uma distância fixa, enquanto **moveHorizontal** permite especificar a distância desejada.

Nota: um pixel é um único ponto da tela. Sua tela é formada por uma grade de pixels individuais.

Figura 1.3 - Menu pop-up de um objeto, listando suas operações.
Figura 1.4 - Desenho de um círculo.

Conceito: métodos podem ter parâmetros para fornecer informações adicionais necessárias a uma tarefa.

1.5 TIPOS DE DADOS

Os valores adicionais exigidos por alguns métodos são chamados de parâmetros. Um método informa quais tipos de parâmetros exige. Ao chamar, por exemplo, o método `moveHorizontal` mostrado na Figura 1.5, a caixa de diálogo exibe, perto da parte superior, a linha:

`void moveHorizontal(int distance)`

Isso é chamado de cabeçalho do método. O cabeçalho fornece algumas informações sobre o método em questão. A parte entre parênteses, `(int distance)`, contém as informações sobre o parâmetro necessário. Para cada parâmetro, são definidos um tipo e um nome. O cabeçalho acima afirma que o método exige um parâmetro do tipo `int` chamado `distance`. O nome dá uma pista sobre o significado do dado esperado. Em conjunto, o nome do método e os tipos de parâmetros encontrados em seu cabeçalho são chamados de assinatura do método.

1.5 Tipos de dados

Um tipo especifica que tipo de dado pode ser passado para um parâmetro. O tipo `int` representa números inteiros.

No exemplo anterior, a assinatura de `moveHorizontal` informa que, antes de o método ser executado, precisamos fornecer um número inteiro especificando a distância do deslocamento. O campo de entrada mostrado na Figura 1.5 permite digitar esse número.

Nos exemplos apresentados até aqui, o único tipo de dado que vimos foi `int`. Os parâmetros dos métodos de movimento e do método `changeSize` são todos desse tipo.

Ao observar com mais atenção o menu pop-up do objeto, vemos que as entradas dos métodos incluem os tipos de seus parâmetros. Se um método não tiver parâmetros, seu nome é seguido por um par de parênteses vazio. Se tiver um parâmetro, o tipo e o nome desse parâmetro são mostrados.

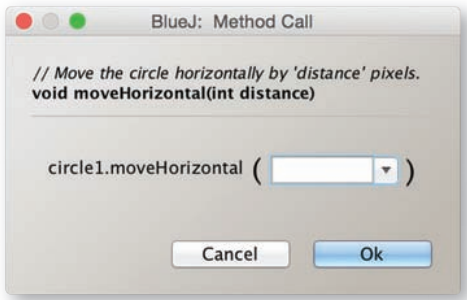
Figura 1.5 - Caixa de diálogo de chamada de método.

Exercício 1.3 - Antes de continuar a leitura, tente invocar os métodos `moveVertical`, `slowMoveVertical` e `changeSize`. Descubra como usar `moveHorizontal` para mover o círculo 70 pixels para a esquerda.

Conceito: o nome do método e os tipos de seus parâmetros formam sua assinatura. A assinatura fornece as informações necessárias para invocar o método.

Conceito: parâmetros possuem tipos. O tipo define quais valores um parâmetro pode receber.

Figure 1.5
A method-call dialog



Exercise 1.3 Try invoking the `moveVertical`, `slowMoveVertical`, and `changeSize` methods before you read on. Find out how you can use `moveHorizontal` to move the circle 70 pixels to the left.

Concept

The method name and the parameter types of a method are called its **signature**. They provide the information needed to invoke that method.

The additional values that some methods require are called *parameters*. A method indicates what kinds of parameters it requires. When calling, for example, the `moveHorizontal` method as shown in Figure 1.5, the dialog displays the line near the top.

`void moveHorizontal(int distance)`

This is called the *header* of the method. The header provides some information about the method in question. The part enclosed by parentheses `(int distance)` is the information about the required parameter. For each parameter, it defines a *type* and a *name*. The header above states that the method requires one parameter of type `int` named `distance`. The name gives a hint about the meaning of the data expected. Together, the name of a method and the parameter types found in its header are called the method's *signature*.

1.5 Data types

Concept

Parameters have **types**. The type defines what kinds of values a parameter can take.

A type specifies what kind of data can be passed to a parameter. The type `int` signifies whole numbers (also called “integer” numbers, hence the abbreviation “int”).

In the example above, the signature of the `moveHorizontal` method states that, before the method can execute, we need to supply a whole number specifying the distance to move. The data entry field shown in Figure 1.5 then lets you enter that number.

In the examples so far, the only data type we have seen has been `int`. The parameters of the move methods and the `changeSize` method are all of that type.

Closer inspection of the object's pop-up menu shows that the method entries in the menu include the parameter types. If a method has no parameter, the method name is followed by an empty set of parentheses. If it has a parameter, the type and name of that parameter

is displayed. In the list of methods for a circle, you will see one method with a different parameter type: the **changeColor** method has a parameter of type **String**.

The **String** type indicates that a section of text (for example, a word or a sentence) is expected. Strings are always enclosed within double quotes. For example, to enter the word *red* as a string, type

`"red"`

The method-call dialog also includes a section of text called a *comment* above the method header. Comments are included to provide information to the (human) reader and are described in Chapter 2. The comment of the **changeColor** method describes what color names the system knows about.

Exercise 1.4 Invoke the **changeColor** method on one of your circle objects and enter the string `"red"`. This should change the color of the circle. Try other colors.

Exercise 1.5 This is a very simple example, and not many colors are supported. See what happens when you specify a color that is not known.

Exercise 1.6 Invoke the **changeColor** method, and write the color into the parameter field *without* the quotes. What happens?

Pitfall A common error for beginners is to forget the double quotes when typing in a data value of type **String**. If you type **green** instead of `"green"`, you will get an error message saying something like "Error: cannot find symbol - variable green."

Java supports several other data types, including decimal numbers and characters. We shall not discuss all of them right now, but rather come back to this issue later. If you want to find out about them now, see at Appendix B.

1.6 Multiple instances

Once you have a class, you can create as many objects (or instances) of that class as you like. From the class **Circle**, you can create many circles. From **Square**, you can create many squares.

Exercise 1.7 Create several circle objects on the object bench. You can do so by selecting **new Circle()** from the pop-up menu of the **Circle** class. Make them visible, then move them around on the screen using the "move" methods. Make one big and yellow; make another one small and green. Try the other shapes too: create a few triangles, squares, and persons. Change their positions, sizes, and colors.

Tradução PT-BR - página original 37

CAPÍTULO 1 - OBJETOS E CLASSES

Na lista de métodos de um círculo, você verá um método com um tipo de parâmetro diferente: **changeColor** possui um parâmetro do tipo **String**.

O tipo **String** indica que se espera um trecho de texto, como uma palavra ou uma frase. Strings são sempre colocadas entre aspas duplas. Por exemplo, para inserir a palavra *red* como uma string, digite:

`"red"`

A caixa de diálogo de chamada de método também inclui, acima do cabeçalho, um trecho de texto chamado comentário. Comentários são incluídos para fornecer informações ao leitor humano e serão descritos no Capítulo 2. O comentário do método **changeColor** descreve quais nomes de cores o sistema reconhece.

Java oferece suporte a vários outros tipos de dados, incluindo números decimais e caracteres. Não discutiremos todos agora; voltaremos a esse assunto mais tarde. Se quiser conhecê-los desde já, consulte o Apêndice B.

1.6 Múltiplas instâncias

Depois que você possui uma classe, pode criar quantos objetos - ou instâncias - dessa classe desejar. A partir da classe **Circle**, pode criar muitos círculos. A partir de **Square**, pode criar muitos quadrados.

Armadilha: um erro comum entre iniciantes é esquecer as aspas duplas ao digitar um valor do tipo **String**. Se você digitar *green* em vez de `"green"`, receberá uma mensagem de erro semelhante a: "Error: cannot find symbol - variable green."

Exercício 1.4 - Invoque o método **changeColor** em um de seus objetos círculo e forneça a string `"red"`. Isso deverá alterar a cor do círculo. Experimente outras cores.

Exercício 1.5 - Este é um exemplo muito simples e poucas cores são aceitas. Veja o que acontece quando você especifica uma cor desconhecida.

Exercício 1.6 - Invoque **changeColor** e escreva a cor no campo do parâmetro sem as aspas. O que acontece?

Exercício 1.7 - Crie vários objetos círculo na bancada de objetos selecionando **new Circle()** no menu da classe **Circle**. Torne-os visíveis e mova-os pela tela usando os métodos de movimento. Faça um deles grande e amarelo; faça outro pequeno e verde. Experimente também as outras formas: crie alguns triângulos, quadrados e pessoas. Altere suas posições, tamanhos e cores.

Concept

Multiple instances.
Many similar objects can be created from a single class.

Every one of those objects has its own position, color, and size. You change an attribute of an object (such as its size) by calling a method on that object. This will affect this particular object, but not others.

You may also notice an additional detail about parameters. Have a look at the `changeSize` method of the triangle. Its header is

```
void changeSize(int newHeight, int newWidth)
```

Here is an example of a method with more than one parameter. This method has two, and a comma separates them in the header. Methods can, in fact, have any number of parameters.

1.7 State

Concept

Objects have state. The **state** is represented by storing values in fields.

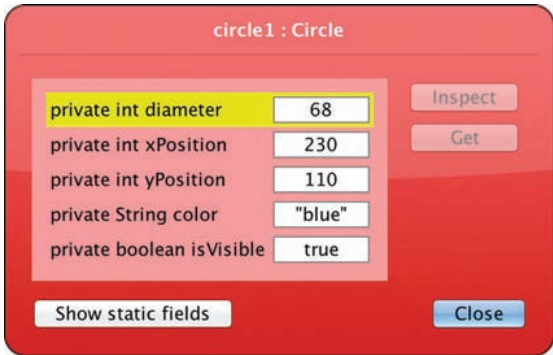
The set of values of all attributes defining an object (such as *x*-position, *y*-position, color, diameter, and visibility status for a circle) is also referred to as the object's *state*. This is another example of common terminology that we shall use from now on.

In BlueJ, the state of an object can be inspected by selecting the *Inspect* function from the object's pop-up menu. When an object is inspected, an *object inspector* is displayed. The object inspector is an enlarged view of the object that shows the attributes stored inside it (Figure 1.6).

Exercise 1.8 Make sure you have several objects on the object bench, and then inspect each of them in turn. Try changing the state of an object (for example, by calling the `moveLeft` method) while the object inspector is open. You should see the values in the object inspector change.

Some methods, when called, change the state of an object. For example, `moveLeft` changes the `xPosition` attribute. Java refers to these object attributes as *fields*.

Figure 1.6
An object inspector, showing details of an object



1.7 ESTADO

Cada um desses objetos possui sua própria posição, cor e tamanho. Você altera um atributo de um objeto, como seu tamanho, chamando um método nesse objeto. Isso afetará esse objeto específico, mas não os demais.

Você também pode observar outro detalhe a respeito dos parâmetros. Veja o método `changeSize` do triângulo. Seu cabeçalho é:

```
void changeSize(int newHeight, int newWidth)
```

Este é um exemplo de método com mais de um parâmetro. Ele possui dois, separados por vírgula no cabeçalho. Na verdade, métodos podem ter qualquer quantidade de parâmetros.

1.7 Estado

O conjunto dos valores de todos os atributos que definem um objeto - como posição *x*, posição *y*, cor, diâmetro e estado de visibilidade de um círculo - também é chamado de estado do objeto. Esse é outro termo comum que passaremos a utilizar.

No BlueJ, o estado de um objeto pode ser examinado escolhendo a função *Inspect* em seu menu pop-up. Quando um objeto é inspecionado, é exibido um inspetor de objetos. O inspetor é uma visualização ampliada do objeto que mostra os atributos armazenados em seu interior (Figura 1.6).

Conceito - múltiplas instâncias: muitos objetos semelhantes podem ser criados a partir de uma única classe.

Conceito - estado: objetos possuem estado. O estado é representado pelo armazenamento de valores em campos.

Exercício 1.8 - Certifique-se de que existam vários objetos na bancada de objetos e inspecione cada um deles. Tente alterar o estado de um objeto - por exemplo, chamando `moveLeft` - enquanto o inspetor estiver aberto. Você deverá observar a alteração dos valores exibidos.

Figura 1.6 - Um inspetor de objetos mostrando detalhes de um objeto.

Alguns métodos, quando chamados, modificam o estado de um objeto. Por exemplo, `moveLeft` altera o atributo `xPosition`. Em Java, esses atributos dos objetos são chamados de campos.

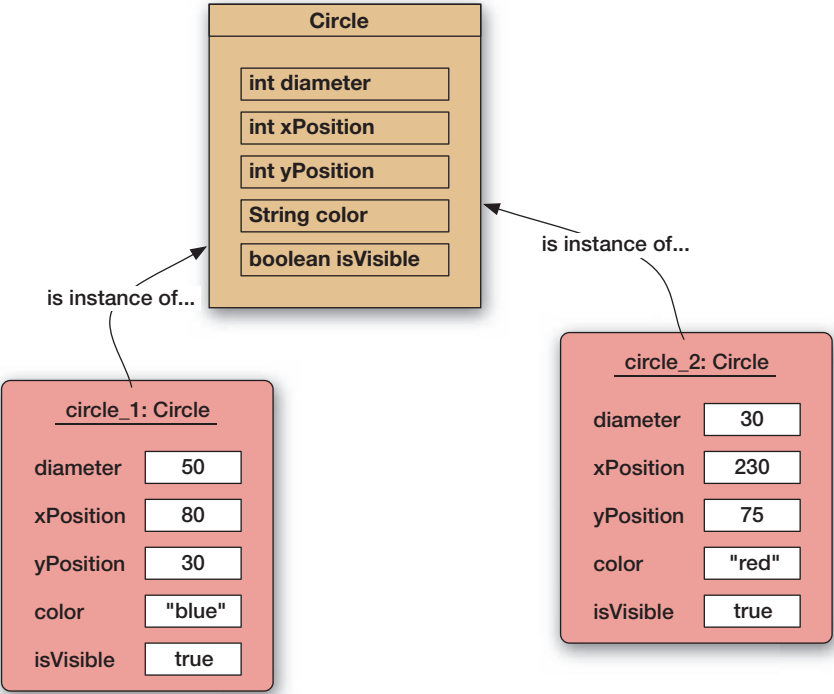
1.8 What is in an object?

On inspecting different objects, you will notice that objects of the *same* class all have the same fields. That is, the number, type, and names of the fields are the same, while the actual value of a particular field in each object may be different. In contrast, objects of a *different* class may have different fields. A circle, for example, has a “diameter” field, while a triangle has fields for “width” and “height.”

The reason is that the number, types, and names of fields are defined in a class, not in an object. So the class **Circle** defines that each circle object will have five fields, named **diameter**, **xPosition**, **yPosition**, **color**, and **isVisible**. It also defines the types for these fields. That is, it specifies that the first three are of type **int**, while the color is of type **String** and the **isVisible** flag is of type **boolean**. (**boolean** is a type that can represent two values: **true** and **false**. We shall discuss it in more detail later.)

When an object of class **Circle** is created, the object will automatically have these fields. The values of the fields are stored in the object. That ensures that each circle has a color, for instance, and each can have a different color (Figure 1.7).

Figure 1.7
A class and its
objects with fields
and values



1.8 O QUE HÁ EM UM OBJETO?

Ao inspecionar objetos diferentes, você perceberá que objetos da mesma classe possuem os mesmos campos. Ou seja, a quantidade, os tipos e os nomes dos campos são iguais, embora o valor efetivo de um determinado campo possa ser diferente em cada objeto. Em contraste, objetos de classes diferentes podem possuir campos diferentes. Um círculo, por exemplo, possui um campo **diameter**, enquanto um triângulo possui campos **width** e **height**.

Isso acontece porque a quantidade, os tipos e os nomes dos campos são definidos em uma classe, e não em cada objeto. Assim, a classe **Circle** define que cada objeto círculo terá cinco campos: **diameter**, **xPosition**, **yPosition**, **color** e **isVisible**. Ela também define os tipos desses campos. Os três primeiros são do tipo **int**, **color** é do tipo **String** e o indicador **isVisible** é do tipo **boolean**. O tipo **boolean** pode representar dois valores, **true** e **false**; ele será discutido em mais detalhes posteriormente.

Quando um objeto da classe **Circle** é criado, ele recebe automaticamente esses campos. Os valores são armazenados no próprio objeto. Isso garante, por exemplo, que cada círculo tenha uma cor e que círculos diferentes possam possuir cores diferentes (Figura 1.7).

Figura 1.7 - Uma classe e seus objetos, com campos e valores.

Os identificadores e valores mostrados no diagrama - **Circle**, **circle_1**, **circle_2**, **diameter**, **xPosition**, **yPosition**, **color**, **isVisible**, **int**, **String**, **boolean**, **true**, **"blue"** e **"red"** - são mantidos como no original por corresponderem a nomes de código e valores Java.

1.9 CÓDIGO JAVA

A história é semelhante para os métodos. Os métodos são definidos na classe do objeto. Como consequência, todos os objetos de uma determinada classe possuem os mesmos métodos. Entretanto, os métodos são invocados em objetos. Isso deixa claro qual objeto deve ser alterado quando, por exemplo, o método `moveRight` é invocado.

Figura 1.8 - Duas imagens criadas a partir de um conjunto de objetos de formas.

Exercício 1.9 - A Figura 1.8 mostra duas imagens diferentes. Escolha uma delas e recrie-a usando as formas do projeto *figures*. Enquanto faz isso, anote o que precisa fazer para obter o resultado. Seria possível fazê-lo de maneiras diferentes?

1.9 Código Java

Quando programamos em Java, basicamente escrevemos instruções para invocar métodos em objetos, assim como fizemos de forma interativa com os objetos de figuras. Entretanto, em vez de escolher métodos em um menu com o mouse, digitamos os comandos em formato textual. Podemos observar como esses comandos aparecem em texto usando o Terminal do BlueJ.

Exercício 1.10 - Selecione *Show Terminal* no menu *View*. Isso abre outra janela usada pelo BlueJ para saída de texto. Depois, selecione *Record method calls* no menu *Options* do terminal. Essa função fará com que todas as chamadas de métodos, em sua forma textual, sejam registradas no terminal. Agora crie alguns objetos, chame alguns de seus métodos e observe a saída na janela do terminal.

Usando a função *Record method calls* do terminal, podemos ver que a sequência para criar um objeto `Person` e chamar seus métodos `makeVisible` e `moveRight` aparece em Java da seguinte forma:

```
Person person1 = new Person();
person1.makeVisible();
person1.moveRight();
```

O código Java é mantido sem tradução.

Figure 1.8
Two images created
from a set of shape
objects



The story is similar for methods. Methods are defined in the class of the object. As a result, all objects of a given class have the same methods. However, the methods are invoked on objects. This makes it clear which object to change when, for example, a `moveRight` method is invoked.

Exercise 1.9 Figure 1.8 shows two different images. Choose one of these images and recreate it using the shapes from the *figures* project. While you are doing this, write down what you have to do to achieve this. Could it be done in different ways?

1.9 **Java code**

When we program in Java, we essentially write down instructions to invoke methods on objects, just as we have done with our figure objects above. However, we do not do this interactively by choosing methods from a menu with the mouse, but instead we type the commands down in textual form. We can see what those commands look like in text form by using the BlueJ Terminal.

Exercise 1.10 Select *Show Terminal* from the *View* menu. This shows another window that BlueJ uses for text output. Then select *Record method calls* from the terminal's *Options* menu. This function will cause all our method calls (in their textual form) to be written to the terminal. Now create a few objects, call some of their methods, and observe the output in the terminal window.

Using the terminal's *Record method calls* function, we can see that the sequence of creating a person object and calling its `makeVisible` and `moveRight` methods looks like this in Java text form:

```
Person person1 = new Person();
person1.makeVisible();
person1.moveRight();
```

Here, we can observe several things:

- We can see what creating an object and giving it a name looks like. Technically, what we are doing here is *storing the Person object into a variable*; we will discuss this in detail in the next chapter.
- We see that, to call a method on an object, we write the name of the object, followed by a dot, followed by the name of the method. The command ends with a parameter list—an empty pair of parentheses if there are no parameters.
- All Java statements end with a semicolon.

Instead of just looking at Java statements, we can also type them. To do this, we use the *Code Pad*. (You can switch off the *Record method calls* function now and close the terminal.)

Exercise 1.11 Select *Show Code Pad* from the *View* menu. This should display a new pane next to the object bench in your main BlueJ window. This pane is the *Code Pad*. You can type Java code here.

In the *Code Pad*, we can type Java code that does the same things we did interactively before. The Java code we need to write is exactly like that shown above.

Exercise 1.12 In the *Code Pad*, type the code shown above to create a *Person* object and call its **makeVisible** and **moveRight** methods. Then go on to create some other objects and call their methods.

Tip

You can recall previously used commands in the *Code Pad* by using the up arrow.

Typing these commands should have the same effect as invoking the same command from the object’s menu. If instead you see an error message, then you have mistyped the command. Check your spelling. You will note that getting even a single character wrong will cause the command to fail.

1.10

Object interaction

For the next section, we shall work with a different example project. Close the *figures* project if you still have it open, and open the project called *house*.

- Exercise 1.13** Open the *house* project. Create an instance of class **Picture** and invoke its **draw** method. Also, try out the **setBlackAndWhite** and **setColor** methods.
- Exercise 1.14** How do you think the **Picture** class draws the picture?

CAPÍTULO 1 - OBJETOS E CLASSES

Aqui podemos observar várias coisas:

- Podemos ver como é criar um objeto e dar um nome a ele. Tecnicamente, o que fazemos aqui é armazenar o objeto *Person* em uma variável; discutiremos isso em detalhes no próximo capítulo.
- Para chamar um método em um objeto, escrevemos o nome do objeto, seguido por um ponto e então pelo nome do método. O comando termina com uma lista de parâmetros - um par de parênteses vazio se não houver parâmetros.
- Todas as instruções Java terminam com ponto e vírgula.

Em vez de apenas observar instruções Java, também podemos digitá-las. Para isso usamos o *Code Pad*. Você pode agora desativar a função *Record method calls* e fechar o terminal.

Exercício 1.11 - Selecione *Show Code Pad* no menu *View*. Isso deverá exibir um novo painel ao lado da bancada de objetos na janela principal do BlueJ. Esse painel é o *Code Pad*. Você pode digitar código Java nele.

No *Code Pad*, podemos escrever código Java que realiza as mesmas ações que fizemos anteriormente de forma interativa. O código Java que precisamos escrever é exatamente como o mostrado anteriormente.

Exercício 1.12 - No *Code Pad*, digite o código mostrado anteriormente para criar um objeto *Person* e chamar seus métodos *makeVisible* e *moveRight*. Em seguida, crie outros objetos e chame seus métodos.

Digitar esses comandos deve produzir o mesmo efeito de invocar o mesmo comando pelo menu do objeto. Se aparecer uma mensagem de erro, você digitou o comando incorretamente. Verifique a escrita. Observe que errar até mesmo um único caractere fará o comando falhar.

1.10 Interação entre objetos

Na próxima seção trabalharemos com outro projeto de exemplo. Feche o projeto *figures*, caso ainda esteja aberto, e abra o projeto chamado *house*.

Dica: você pode recuperar comandos usados anteriormente no *Code Pad* pressionando a seta para cima.

Exercício 1.13 - Abra o projeto *house*. Crie uma instância da classe *Picture* e invoque seu método *draw*. Experimente também os métodos *setBlackAndWhite* e *setColor*.

Exercício 1.14 - Como você acha que a classe *Picture* desenha a figura?

Five of the classes in the *house* project are identical to the classes in the *figures* project. But we now have an additional class: **Picture**. This class is programmed to do exactly what we have done by hand in Exercise 1.9.

In reality, if we want a sequence of tasks done in Java, we would not normally do it by hand. Instead, we would create a class that does it for us. This is the **Picture** class.

The **Picture** class is written so that, when you create an instance, the instance creates two square objects (one for the wall, one for the window), a triangle, and a circle. Then, when you call the draw method, it moves them around and changes their color and size, until the canvas looks like the picture we see in Figure 1.8.

The important points here are that: objects can create other objects; and they can call each other's methods. In a normal Java program, you may well have hundreds or thousands of objects. The user of a program just starts the program (which typically creates a first object), and all other objects are created—directly or indirectly—by that object.

The big question now is this: How do we write the class for such an object?

Concept

Method calling. Objects can communicate by calling each other's methods.

1.11 Source code

Concept

The **source code** of a class determines the structure and behavior (the fields and methods) of each of the objects of that class.

Each class has some *source code* associated with it. The source code is text that defines the details of the class. In BlueJ, the source code of a class can be viewed by selecting the *Open Editor* function from the class's pop-up menu, or by double-clicking the class icon.

Exercise 1.15 Look at the pop-up menu of class **Picture** again. You will see an option labeled *Open Editor*. Select it. This will open a text editor displaying the source code of the class.

The source code is text written in the Java programming language. It defines what fields and methods a class has, and precisely what happens when a method is invoked. In the next chapter, we shall discuss exactly what the source code of a class contains, and how it is structured.

A large part of learning the art of programming is learning how to write these class definitions. To do this, we shall learn to use the Java language (although there are many other programming languages that could be used to write code).

When you make a change to the source code and close the editor, the icon for that class appears striped in the diagram.³ The stripes indicate that the source has been changed. The class now needs to be compiled by clicking the *Compile* button. (You may like to read the “About compilation” note for more information on what is happening when you compile a class.) Once a class has been compiled, objects can be created again and you can try out your change.

³ In BlueJ, there is no need to explicitly save the text in the editor before closing. If you close the editor, the source code will automatically be saved.

1.11 CÓDIGO-FONTE

Cinco das classes do projeto house são idênticas às classes do projeto figures. Agora, porém, temos uma classe adicional: Picture. Essa classe foi programada para fazer exatamente aquilo que fizemos manualmente no Exercício 1.9.

Na prática, quando queremos executar uma sequência de tarefas em Java, normalmente não fazemos tudo manualmente. Em vez disso, criamos uma classe que faça isso por nós. Essa é a classe Picture.

A classe Picture foi escrita de modo que, quando criamos uma instância, ela cria dois objetos quadrados - um para a parede e outro para a janela -, um triângulo e um círculo. Depois, quando chamamos o método draw, ela os move e altera suas cores e tamanhos até que a tela tenha a aparência da figura mostrada na Figura 1.8.

Os pontos importantes aqui são: objetos podem criar outros objetos; e objetos podem chamar os métodos uns dos outros. Em um programa Java normal, podemos ter centenas ou milhares de objetos. O usuário apenas inicia o programa - que normalmente cria um primeiro objeto - e todos os demais objetos são criados, direta ou indiretamente, por esse objeto.

A grande pergunta agora é: como escrevemos a classe de um objeto desse tipo?

1.11 Código-fonte

Cada classe possui um código-fonte associado. O código-fonte é um texto que define os detalhes da classe. No BlueJ, o código-fonte de uma classe pode ser visualizado selecionando Open Editor no menu pop-up da classe ou dando um clique duplo no ícone da classe.

Conceito - chamada de métodos: objetos podem se comunicar chamando os métodos uns dos outros.

Exercício 1.15 - Observe novamente o menu pop-up da classe Picture. Você verá uma opção chamada Open Editor. Selecione-a. Um editor de texto será aberto exibindo o código-fonte da classe.

O código-fonte é escrito na linguagem de programação Java. Ele define quais campos e métodos uma classe possui e exatamente o que acontece quando um método é invocado. No próximo capítulo discutiremos exatamente o que o código-fonte de uma classe contém e como ele é estruturado.

Uma grande parte da aprendizagem da arte da programação consiste em aprender a escrever essas definições de classes. Para isso, aprenderemos a utilizar a linguagem Java, embora existam muitas outras linguagens de programação que também poderiam ser utilizadas para escrever código.

Quando você altera o código-fonte e fecha o editor, o ícone da classe aparece listrado no diagrama. As listras indicam que o código-fonte foi alterado. A classe então precisa ser compilada clicando no botão Compile. Depois que a classe for compilada, objetos poderão ser criados novamente e você poderá testar sua alteração.

Nota: no BlueJ não é necessário salvar explicitamente o texto no editor antes de fechá-lo. Ao fechar o editor, o código-fonte é salvo automaticamente.

Conceito: o código-fonte de uma classe determina a estrutura e o comportamento - campos e métodos - de cada objeto dessa classe.

About compilation When people write computer programs, they typically use a “higher-level” programming language such as Java. A problem with that is that a computer cannot execute Java source code directly. Java was designed to be reasonably easy to read for humans, not for computers. Computers, internally, work with a binary representation of a machine code, which looks quite different from Java. The problem for us is that it looks so complex that we do not want to write it directly. We prefer to write Java. What can we do about this?

The solution is a program called the *compiler*. The compiler translates the Java code into machine code. We can write Java and run the compiler—which generates the machine code—and the computer can then read the machine code. As a result, every time we change the source code, we must first run the compiler before we can use the class again to create an object. Otherwise, the machine code version that the computer needs will not exist.

Exercise 1.16 In the source code of class **Picture**, find the part that actually draws the picture. Change it so that the sun will be blue rather than yellow.

Exercise 1.17 Add a second sun to the picture. To do this, pay attention to the field definitions close to the top of the class. You will find this code:

```
private Square wall;  
private Square window;  
private Triangle roof;  
private Circle sun;
```

You need to add a line here for the second sun. For example:

```
private Circle sun2;
```

Then write the appropriate code in two different places for creating the second sun and making it visible when the picture is drawn.

Exercise 1.18 *Challenge exercise* (This means that this exercise might not be solved quickly. We do not expect everyone to be able to solve this at the moment. If you do, great. If you don’t, then don’t worry. Things will become clearer as you read on. Come back to this exercise later.) Add a sunset to the single-sun version of **Picture**. That is, make the sun go down slowly. Remember: The circle has a method **slowMoveVertical** that you can use to do this.

Exercise 1.19 *Challenge exercise* If you added your sunset to the end of the **draw** method (so that the sun goes down automatically when the picture is drawn), change this now. We now want the sunset in a separate method, so that we can call **draw** and see the picture with the sun up, and then call **sunset** (a separate method!) to make the sun go down.

CAPÍTULO 1 - OBJETOS E CLASSES

Sobre compilação

Quando as pessoas escrevem programas de computador, normalmente utilizam uma linguagem de programação de “alto nível”, como Java. O problema é que o computador não consegue executar diretamente o código-fonte Java. Java foi projetada para ser razoavelmente fácil de ler por seres humanos, e não por computadores. Internamente, os computadores trabalham com uma representação binária de código de máquina, muito diferente de Java. Para nós, o problema é que esse código é tão complexo que não queremos escrevê-lo diretamente. Preferimos escrever Java. O que podemos fazer?

A solução é um programa chamado compilador. O compilador traduz o código Java para uma forma executável pela máquina. Podemos escrever Java e executar o compilador, que gera o código necessário para que o computador execute o programa. Como consequência, sempre que modificamos o código-fonte, precisamos executar o compilador antes de voltar a usar a classe para criar um objeto. Caso contrário, a versão de código necessária ao computador não existirá ou estará desatualizada.

Exercício 1.16 - No código-fonte da classe **Picture**, encontre a parte que realmente desenha a figura. Altere-a para que o sol seja azul em vez de amarelo.

Exercício 1.17 - Adicione um segundo sol à figura. Para isso, observe as definições de campos próximas ao início da classe. Você encontrará o seguinte código:

```
private Square wall;  
private Square window;  
private Triangle roof;  
private Circle sun;
```

Você precisa acrescentar uma linha para o segundo sol, por exemplo:

```
private Circle sun2;
```

Depois escreva o código apropriado em dois pontos diferentes: para criar o segundo sol e para torná-lo visível quando a figura for desenhada.

Exercício 1.18 - Exercício-desafio. Isso significa que talvez ele não seja resolvido rapidamente. Não esperamos que todos consigam resolvê-lo neste momento. Se conseguir, ótimo. Se não conseguir, não se preocupe. As coisas ficarão mais claras à medida que você continuar lendo. Volte a este exercício posteriormente. Adicione um pôr do sol à versão de **Picture** com apenas um sol, fazendo o sol descer lentamente. Lembre-se: **Circle** possui o método **slowMoveVertical**, que pode ser usado para isso.

Exercício 1.19 - Exercício-desafio. Se você adicionou o pôr do sol ao final do método **draw**, fazendo o sol descer automaticamente quando a figura é desenhada, altere isso agora. Queremos que o pôr do sol fique em um método separado, para que possamos chamar **draw**, ver a figura com o sol no alto e depois chamar **sunset** - um método separado - para fazer o sol descer.

Exercise 1.20 *Challenge exercise* Make a person walk up to the house after the sunset.

1.12 Another example

In this chapter, we have already discussed a large number of new concepts. To help in understanding these concepts, we shall now revisit them in a different context. For this, we use another example. Close the *house* project if you still have it open, and open the *lab-classes* project.

This project is a simplified part of a student database designed to keep track of students in laboratory classes, and to print class lists.

Exercise 1.21 Create an object of class **Student**. You will notice that this time you are prompted not only for a name of the instance, but also for some other parameters. Fill them in before clicking OK. (Remember that parameters of type **String** must be written within double quotes.)

1.13 Return values

As before, you can create multiple objects. And again, as before, the objects have methods that you can call from their pop-up menu(s).

Exercise 1.22 Create some **Student** objects. Call the **getName** method on each object. Explain what is happening.

Concept

Result. Methods may return information about an object via a **return value**.

When calling the **getName** method of the **Student** class, we notice something new: methods may return a result value. In fact, the header of each method tells us whether or not it returns a result, and what the type of the result is. The header of **getName** (as shown in the object's pop-up menu) is defined as

String getName()

The word **String** before the method name specifies the return type. In this case, it states that calling this method will return a result of type **String**. The header of **changeName** states:

void changeName(String replacementName)

The word **void** indicates that this method does not return any result.

Methods with return values enable us to get information from an object via a method call. This means that we can use methods either to change an object's state, or to find out about its state. The return type of a method is not part of its signature.

1.12 OUTRO EXEMPLO

Neste capítulo já discutimos uma grande quantidade de conceitos novos. Para ajudar na compreensão, vamos agora revisita-los em um contexto diferente. Usaremos outro exemplo. Feche o projeto house, caso ainda esteja aberto, e abra o projeto lab-classes.

Esse projeto é uma parte simplificada de um banco de dados de estudantes, projetado para acompanhar alunos matriculados em turmas de laboratório e imprimir listas de turma.

Exercício 1.20 - Exercício-desafio: faça uma pessoa caminhar até a casa depois do pôr do sol.

Exercício 1.21 - Crie um objeto da classe Student. Você perceberá que, desta vez, além do nome da instância, serão solicitados alguns outros parâmetros. Preencha-os antes de clicar em OK. Lembre-se de que parâmetros do tipo String precisam ser escritos entre aspas duplas.

Exercício 1.22 - Crie alguns objetos Student. Chame o método getName em cada objeto. Explique o que está acontecendo.

1.13 Valores de retorno

Como antes, você pode criar vários objetos. E, novamente, os objetos possuem métodos que podem ser chamados por seus menus pop-up.

Conceito - resultado: métodos podem retornar informações sobre um objeto por meio de um valor de retorno.

Ao chamar o método getName da classe Student, percebemos algo novo: métodos podem retornar um valor de resultado. De fato, o cabeçalho de cada método informa se ele retorna ou não um resultado e qual é o tipo desse resultado. O cabeçalho de getName, conforme aparece no menu do objeto, é:

String getName()

A palavra String antes do nome do método especifica o tipo de retorno. Neste caso, ela indica que chamar o método retornará um resultado do tipo String. O cabeçalho de changeName é:

void changeName(String replacementName)

A palavra void indica que esse método não retorna resultado algum.

Métodos com valores de retorno permitem obter informações de um objeto por meio de uma chamada de método. Assim, podemos usar métodos tanto para alterar o estado de um objeto quanto para descobrir informações sobre seu estado. O tipo de retorno de um método não faz parte de sua assinatura.

1.14 Objects as parameters

Exercise 1.23 Create an object of class **LabClass**. As the signature indicates, you need to specify the maximum number of students in that class (an integer).

Exercise 1.24 Call the **numberOfStudents** method of that class. What does it do?

Exercise 1.25 Look at the signature of the **enrollStudent** method. You will notice that the type of the expected parameter is **Student**. Make sure you have two or three students and a **LabClass** object on the object bench, then call the **enrollStudent** method of the **LabClass** object. With the input cursor in the dialog entry field, click on one of the student objects; this enters the name of the student object into the parameter field of the **enrollStudent** method (Figure 1.9). Click OK and you have added the student to the **LabClass**. Add one or more other students as well.

Exercise 1.26 Call the **printList** method of the **LabClass** object. You will see a list of all the students in that class printed to the BlueJ terminal window (Figure 1.10).

Figure 1.9
Adding a student to
a **LabClass**

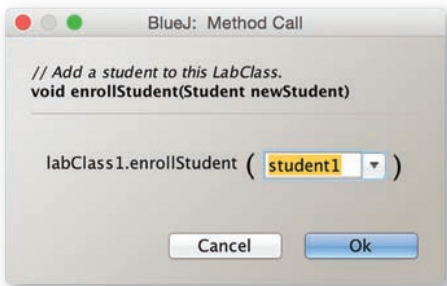
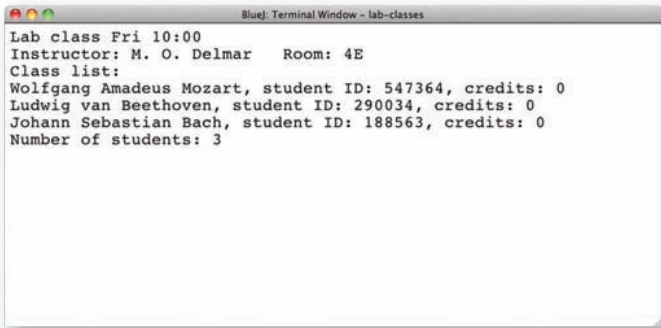


Figure 1.10
Output of the
LabClass class
listing



1.14 OBJETOS COMO PARÂMETROS

Figura 1.9 - Adicionando um estudante a uma LabClass.

Exercício 1.23 - Crie um objeto da classe LabClass. Como a assinatura indica, você precisa especificar o número máximo de estudantes da turma, usando um número inteiro.

Exercício 1.24 - Chame o método numberOfStudents dessa classe. O que ele faz?

Exercício 1.25 - Observe a assinatura do método enrollStudent. Você verá que o tipo do parâmetro esperado é Student. Certifique-se de ter dois ou três estudantes e um objeto LabClass na bancada de objetos. Em seguida, chame enrollStudent no objeto LabClass. Com o cursor de entrada no campo da caixa de diálogo, clique em um dos objetos de estudante; isso insere o nome desse objeto no campo do parâmetro de enrollStudent, como mostra a Figura 1.9. Clique em OK e o estudante será adicionado à LabClass. Adicione também um ou mais estudantes.

Exercício 1.26 - Chame o método printList do objeto LabClass. Você verá uma lista de todos os estudantes da turma impressa na janela do terminal do BlueJ, como na Figura 1.10.

Figura 1.10 - Saída da listagem produzida pela classe LabClass.

As the exercises show, objects can be passed as parameters to methods of other objects. In the case where a method expects an object as a parameter, the expected object’s class name is specified as the parameter type in the method signature.

Explore this project a bit more. Try to identify the concepts discussed in the *figures* example in this context.

Exercise 1.27 Create three students with the following details:
Snow White, student ID: *A00234*, credits: *24*
Lisa Simpson, student ID: *C22044*, credits: *56*
Charlie Brown, student ID: *A12003*, credits: *6*
Then enter all three into a lab and print a list to the screen.

Exercise 1.28 Use the inspector on a **LabClass** object to discover what fields it has.

Exercise 1.29 Set the instructor, room, and time for a lab, and print the list to the terminal window to check that these new details appear.

1.15

Summary

In this chapter, we have explored the basics of classes and objects. We have discussed the fact that objects are specified by classes. Classes represent the general concept of things, while objects represent concrete instances of a class. We can have many objects of any class.

Objects have methods that we use to communicate with them. We can use a method to make a change to the object or to get information from the object. Methods can have parameters, and parameters have types. Methods have return types, which specify what type of data they return. If the return type is **void**, they do not return anything.

Objects store data in fields (which also have types). All the data values of an object together are referred to as the object’s state.

Objects are created from class definitions that have been written in a particular programming language. Much of programming in Java is about learning to write class definitions. A large Java program will have many classes, each with many methods that call each other in many different ways.

To learn to develop Java programs, we need to learn how to write class definitions, including fields and methods, and how to put these classes together well. The rest of this book deals with these issues.

Terms introduced in this chapter:

object, class, instance, method, signature, parameter, type, state, source code, return value, compiler

1.15 RESUMO

Como mostram os exercícios, objetos podem ser passados como parâmetros para métodos de outros objetos. Quando um método espera receber um objeto como parâmetro, o nome da classe desse objeto é especificado como tipo do parâmetro na assinatura do método.

Explore um pouco mais esse projeto. Tente identificar, nesse contexto, os conceitos discutidos no exemplo *figures*.

Exercício 1.27 - Crie três estudantes com os seguintes dados:
Snow White, ID de estudante: A00234, créditos: 24
Lisa Simpson, ID de estudante: C22044, créditos: 56
Charlie Brown, ID de estudante: A12003, créditos: 6
Depois insira os três em uma turma de laboratório e imprima a lista na tela.

Exercício 1.28 - Use o inspetor em um objeto LabClass para descobrir quais campos ele possui.

Exercício 1.29 - Defina professor, sala e horário de uma turma de laboratório e imprima a lista no terminal para verificar se os novos detalhes aparecem.

1.15 Resumo

Neste capítulo exploramos os fundamentos de classes e objetos. Discutimos que objetos são especificados por classes. As classes representam o conceito geral das coisas, enquanto objetos representam instâncias concretas de uma classe. Podemos ter muitos objetos de qualquer classe.

Objetos possuem métodos que usamos para nos comunicar com eles. Podemos utilizar um método para alterar o objeto ou para obter informações sobre ele. Métodos podem possuir parâmetros, e parâmetros possuem tipos. Métodos possuem tipos de retorno, que especificam que tipo de dado será devolvido. Se o tipo de retorno for void, o método não retorna nada.

Objetos armazenam dados em campos, que também possuem tipos. O conjunto de todos os valores de dados de um objeto é chamado de estado do objeto.

Objetos são criados a partir de definições de classes escritas em uma linguagem de programação específica. Grande parte da programação em Java consiste em aprender a escrever definições de classes. Um programa Java grande terá muitas classes, cada uma com muitos métodos que chamam uns aos outros de várias maneiras.

Para aprender a desenvolver programas Java, precisamos aprender a escrever definições de classes, incluindo campos e métodos, e a combinar essas classes de maneira adequada. O restante deste livro trata dessas questões.

Termos apresentados neste capítulo:
objeto, classe, instância, método, assinatura, parâmetro, tipo, estado, código-fonte, valor de retorno, compilador.

Concept summary

- **object** Java objects model objects from a problem domain.
- **class** Objects are created from classes. The class describes the kind of object; the objects represent individual instances of the class.
- **method** We can communicate with objects by invoking methods on them. Objects usually do something if we invoke a method.
- **parameter** Methods can have parameters to provide additional information for a task.
- **signature** The method name and the parameter types of a method are called its signature. They provide the information needed to invoke that method.
- **type** Parameters have types. The type defines what kinds of values a parameter can take.
- **multiple instances** Many similar objects can be created from a single class.
- **state** Objects have state. The state is represented by storing values in fields.
- **method calling** Objects can communicate by calling each other's methods.
- **source code** The source code of a class determines the structure and behavior (the fields and methods) of each of the objects of that class.
- **result** Methods may return information about an object via a return value.

Exercise 1.30 In this chapter we have mentioned the data types `int` and `String`. Java has more predefined data types. Find out what they are and what they are used for. To do this, you can check Appendix B, or look it up in another Java book or in an online Java language manual. One such manual is at

`http://download.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html`

Exercise 1.31 What are the types of the following values?

`0`
`"hello"`
`101`
`-1`
`true`
`"33"`
`3.1415`

CAPÍTULO 1 - RESUMO DE CONCEITOS

- objeto: objetos Java modelam objetos de um domínio de problema.
- classe: objetos são criados a partir de classes. A classe descreve o tipo de objeto; os objetos representam instâncias individuais da classe.
- método: podemos nos comunicar com objetos invocando métodos neles. Em geral, os objetos fazem algo quando um método é invocado.
- parâmetro: métodos podem ter parâmetros para fornecer informações adicionais necessárias a uma tarefa.
- assinatura: o nome do método e os tipos de seus parâmetros formam sua assinatura. Eles fornecem as informações necessárias para invocar o método.
- tipo: parâmetros possuem tipos. O tipo define quais valores um parâmetro pode receber.
- múltiplas instâncias: muitos objetos semelhantes podem ser criados a partir de uma única classe.
- estado: objetos possuem estado. O estado é representado pelo armazenamento de valores em campos.
- chamada de métodos: objetos podem se comunicar chamando os métodos uns dos outros.
- código-fonte: o código-fonte de uma classe determina a estrutura e o comportamento - campos e métodos - de cada objeto dessa classe.
- resultado: métodos podem retornar informações sobre um objeto por meio de um valor de retorno.

Exercício 1.30 - Neste capítulo mencionamos os tipos de dados `int` e `String`. Java possui outros tipos predefinidos. Descubra quais são e para que servem. Você pode consultar o Apêndice B, outro livro de Java ou um manual on-line da linguagem. Um deles está em:
`http://download.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html`

Exercício 1.31 - Quais são os tipos dos seguintes valores?

`0`
`"hello"`
`101`
`-1`
`true`
`"33"`
`3.1415`

Exercise 1.32 What would you have to do to add a new field, for example one called `name`, to a circle object?

Exercise 1.33 Write the header for a method named `send` that has one parameter of type `String`, and does not return a value.

Exercise 1.34 Write the header for a method named `average` that has two parameters, both of type `int`, and returns an `int` value.

Exercise 1.35 Look at the book you are reading right now. Is it an object or a class? If it is a class, name some objects. If it is an object, name its class.

Exercise 1.36 Can an object have several different classes? Discuss.

1.15 RESUMO

Exercício 1.32 - O que você precisaria fazer para adicionar um novo campo, por exemplo um campo chamado `name`, a um objeto círculo?

Exercício 1.33 - Escreva o cabeçalho de um método chamado `send` que tenha um parâmetro do tipo `String` e não retorne valor.

Exercício 1.34 - Escreva o cabeçalho de um método chamado `average` que tenha dois parâmetros, ambos do tipo `int`, e retorne um valor `int`.

Exercício 1.35 - Observe o livro que você está lendo neste momento. Ele é um objeto ou uma classe? Se for uma classe, cite alguns objetos. Se for um objeto, diga qual é sua classe.

Exercício 1.36 - Um objeto pode ter várias classes diferentes? Discuta.

Esta página foi deixada intencionalmente em branco.

This page intentionally left blank

CHAPTER

2

Understanding Class Definitions

Main concepts discussed in this chapter:

- fields
- constructors
- parameters
- methods (accessor, mutator)
- assignment and conditional statement

Java constructs discussed in this chapter:

field, constructor, comment, parameter, assignment (=), block, return statement, **void**, compound assignment operators (+=, -=), if-statement

In this chapter, we take our first proper look at the source code of a class. We will discuss the basic elements of class definitions: *fields*, *constructors*, and *methods*. Methods contain statements, and initially we look at methods containing only simple arithmetic and printing statements. Later, we introduce *conditional statements* that allow choices between different actions to be made within methods.

We shall start by examining a new project in a fair amount of detail. This project represents a naïve implementation of an automated ticket machine. As we start by introducing the most basic features of classes, we shall quickly find that this implementation is deficient in a number of ways. So we shall then proceed to describe a more sophisticated version of the ticket machine that represents a significant improvement. Finally, in order to reinforce the concepts introduced in this chapter, we take a look at the internals of the *lab-classes* example encountered in Chapter 1.

2.1 Ticket machines

Train stations often provide ticket machines that print a ticket when a customer inserts the correct money for their fare. In this chapter, we shall define a class that models something like these ticket machines. As we shall be looking inside our first Java example classes, we shall keep our simulation fairly simple to start with. That will give us the

CAPÍTULO 2 - COMPREENDENDO DEFINIÇÕES DE CLASSES

Neste capítulo, teremos nosso primeiro contato adequado com o código-fonte de uma classe. Discutiremos os elementos básicos das definições de classes: campos, construtores e métodos. Métodos contêm instruções e, inicialmente, observaremos métodos que contêm apenas instruções aritméticas simples e instruções de impressão. Mais adiante, apresentaremos instruções condicionais, que permitem escolher entre diferentes ações dentro de um método.

Começaremos examinando em bastante detalhe um novo projeto. Ele representa uma implementação ingênua de uma máquina automática de bilhetes. Como começamos apresentando as características mais básicas das classes, perceberemos rapidamente que essa implementação possui várias deficiências. Em seguida descreveremos uma versão mais sofisticada da máquina de bilhetes, representando uma melhoria significativa. Por fim, para reforçar os conceitos introduzidos neste capítulo, examinaremos o interior do exemplo *lab-classes* encontrado no Capítulo 1.

2.1 Máquinas de bilhetes

Estações de trem frequentemente possuem máquinas que imprimem um bilhete quando o cliente insere a quantia correta para a tarifa. Neste capítulo definiremos uma classe que modela algo semelhante a essas máquinas. Como examinaremos o interior de nossas primeiras classes de exemplo em Java, manteremos a simulação relativamente simples no início.

Principais conceitos discutidos neste capítulo:

- campos;
- construtores;
- parâmetros;
- métodos - acessores e modificadores;
- atribuição e instrução condicional.

Construções Java discutidas neste capítulo:

field, constructor, comment, parameter, assignment (=), block, return statement, void, operadores de atribuição compostos (+=, -=), instrução if.

Os termos que correspondem diretamente à sintaxe e a palavras-chave Java são mantidos no original quando aparecem como código.